

Wykład 6 i 8

Modele języka naturalnego. Przetwarzanie języka naturalnego

NLP-natural language processing

Transformery, LLM

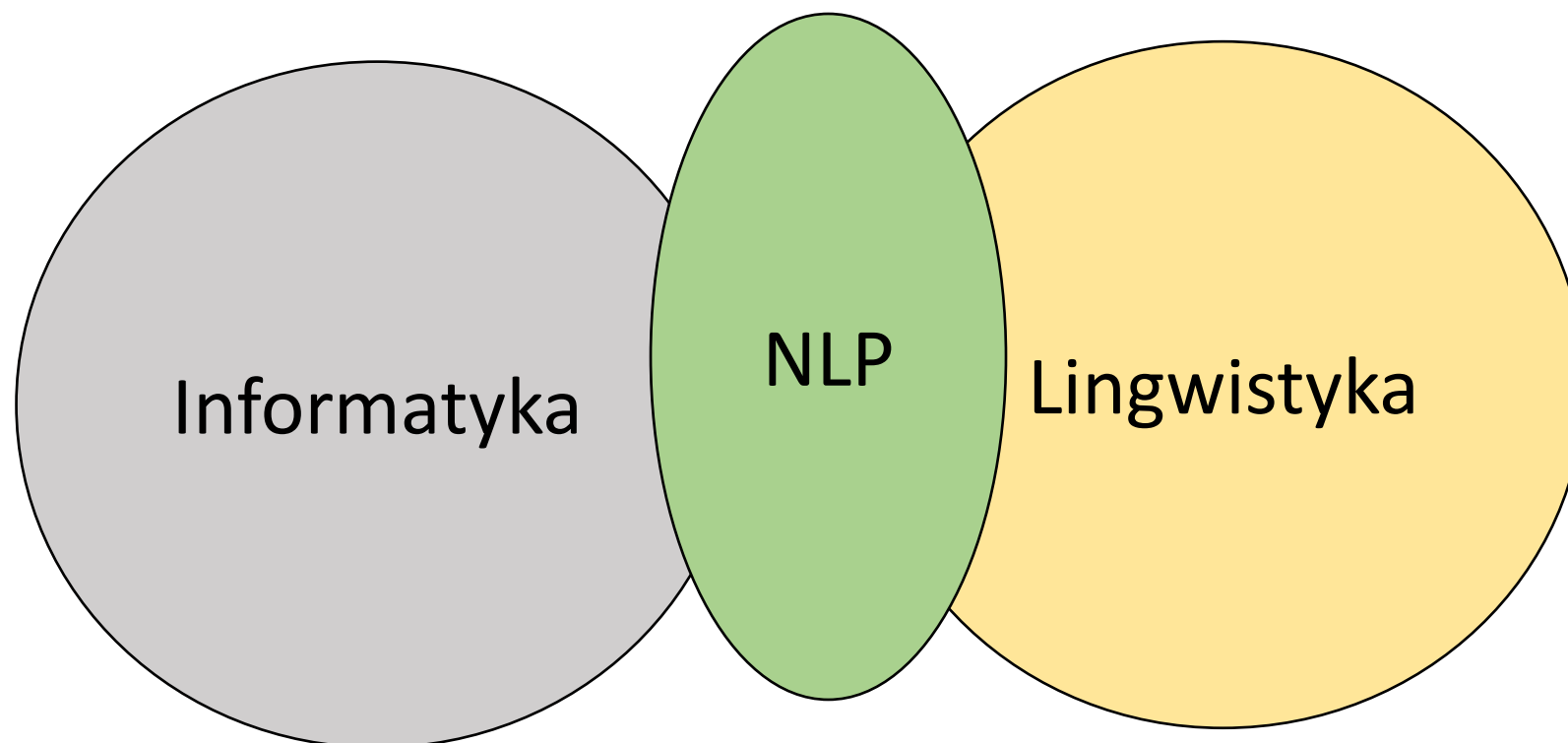
NLP natural language processing

- **Przetwarzanie języka naturalnego (NLP)** to aspekt sztucznej inteligencji, który pomaga komputerom rozumieć, interpretować i wykorzystywać ludzkie języki.
- NLP pozwala komputerom komunikować się z ludźmi, używając ludzkiego języka. Przetwarzanie języka naturalnego zapewnia również komputerom możliwość czytania tekstu, słuchania mowy i interpretowania jej.
- NLP czerpie z kilku dyscyplin, w tym **lingwistyki obliczeniowej** i **informatyki**, próbując wypełnić lukę między komunikacją ludzką a komputerową.

- NLP wykorzystuje zaawansowane technologie, takie jak lingwistyka komputerowa czy zaawansowane modele uczenia maszynowego i głębokiego uczenia, aby umożliwić komputerom przetwarzanie ludzkiego języka – zarówno w formie tekstu, jak i innych form, takich jak nagrania głosowe.
- Głównym celem rozwiązań wykorzystujących NLP jest nie tylko zrozumienie znaczenia poszczególnych słów, ale całej wypowiedzi, uwzględniając przy tym kontekst, intencje, a nawet emocje osoby, której wypowiedź jest przetwarzana.

Definicja (Wikipedia)

Przetwarzanie języka naturalnego (ang. natural language processing, NLP) – interdyscyplinarna dziedzina, łącząca zagadnienia sztucznej inteligencji i językoznawstwa, zajmująca się automatyzacją analizy, rozumienia, tłumaczenia i generowania języka naturalnego przez komputer. System generujący język naturalny przekształca informacje zapisane w bazie danych komputera na język łatwy do odczytania i zrozumienia przez człowieka. Zaś system rozumiejący język naturalny przekształca próbki języka naturalnego na bardziej formalne symbole, łatwiejsze do przetworzenia dla programów komputerowych. Wiele problemów NLP wiąże się zarówno z generacją, jak i rozumieniem języka, np. model morfologiczny zdania (struktura słów), który komputer powinien zbudować, jest potrzebny zarazem do tego, by zdanie było zrozumiałe, jak i gramatycznie poprawne.



Elementy historii

- Twórcą koncepcji „języka jako nauki”, która ostatecznie doprowadziła do przetwarzania języka naturalnego, był szwajcarski profesor lingwistyki Ferdinand de Saussure.
- W latach 1906-1911 profesor Saussure prowadził trzy kursy na Uniwersytecie Genewskim, gdzie opracował podejście opisujące języki jako „systemy”. W ramach języka dźwięk reprezentuje pojęcie, które zmienia znaczenie wraz ze zmianą kontekstu.

Elementy historii

- W 1950 roku Alan Turing napisał artykuł opisujący test na „myślącą” maszynę. Stwierdził, że jeśli maszyna może być częścią rozmowy za pomocą teleprinteru i naśladuje człowieka tak całkowicie, że nie ma zauważalnych różnic, to można uznać, że maszyna jest zdolna do myślenia.
- Wkrótce po tym, w 1952 roku, model Hodgkina-Huxleya pokazał, w jaki sposób mózg wykorzystuje neurony do tworzenia sieci elektrycznej.
- Wydarzenia te pomogły zainspirować ideę sztucznej inteligencji (AI), przetwarzania języka naturalnego (NLP) i ewolucji komputerów.

Elementy historii

Noam Chomsky opublikował **Syntactic Structures** w **1957 roku**. W tej książce zrewolucjonizował koncepcje lingwistyczne i doszedł do wniosku, że aby komputer mógł zrozumieć język, struktura zdania musiałaby zostać zmieniona. Mając to na uwadze, Chomsky stworzył styl gramatyki zwany **gramatyką fazowo-strukturalną**, który metodycznie tłumaczył zdania języka naturalnego na format użyteczny dla komputerów. (Ogólnym celem było stworzenie komputera zdolnego do naśladowania ludzkiego mózgu pod względem myślenia i komunikacji - sztucznej inteligencji).

Elementy historii

- W 1958 roku John McCarthy wydał język programowania LISP (Locator/Identifier Separation Protocol), język komputerowy używany do dziś.
- W 1964 roku opracowano **ELIZA**, proces „pisania na maszynie” komentarzy i odpowiedzi, zaprojektowany w celu naśladowania psychiatri przy użyciu technik refleksji. (Odkazywało się to poprzez przestawianie zdań i przestrzeganie stosunkowo prostych zasad gramatycznych, ale komputer nie był w stanie ich zrozumieć). Również w 1964 roku amerykańska **Narodowa Rada Badawcza (NRC)** utworzyła Komitet Doradczy ds. Automatycznego Przetwarzania Języka (Automatic Language Processing Advisory Committee, w skrócie ALPAC). Zadaniem tego komitetu była ocena postępów w badaniach nad przetwarzaniem języka naturalnego.

Elementy historii

- W 1966 roku NRC i ALPAC zainicjowały pierwsze zatrzymanie AI i NLP, wstrzymując finansowanie badań nad przetwarzaniem języka naturalnego i tłumaczeniem maszynowym.
- Po 12 latach badań i 20 milionach dolarów, tłumaczenia maszynowe były nadal droższe niż ręczne tłumaczenia ludzkie i wciąż nie było komputerów, które zbliżyłyby się do możliwości prowadzenia podstawowej rozmowy.
- W 1966 roku badania nad sztuczną inteligencją i przetwarzaniem języka naturalnego (NLP) były przez wielu (choć nie wszystkich) uważane za ślepy zaułek.

Elementy historii

SHRDLU – system komputerowy z dziedziny sztucznej inteligencji służący do przetwarzania języka naturalnego, napisany przez Terry'ego Winograda w MIT Artificial Intelligence Laboratory w latach 1968–1970. SHRDLU umożliwia prostą konwersację (poprzez terminal) z użytkownikiem na temat małego świata obiektów.

Elementy historii

Przetwarzanie języka naturalnego i badania nad sztuczną inteligencją potrzebowały prawie 14 lat (do 1980 r.), aby otrząsnąć się z niespełnionych oczekiwań stworzonych przez ekstremalnych entuzjastów. Pod pewnymi względami zatrzymanie sztucznej inteligencji zapoczątkowało nową fazę świeżych pomysłów, z porzuceniem wcześniejszych koncepcji tłumaczenia maszynowego i nowymi pomysłami promującymi nowe badania, w tym systemy eksperckie. Mieszanie lingwistyki i statystyki, które było popularne we wczesnych badaniach NLP, zostało zastąpione tematem czystej statystyki. Lata 80. zapoczątkowały fundamentalną reorientację, w której proste przybliżenia zastąpiły głęboką analizę, a proces oceny stał się bardziej rygorystyczny.

Elementy historii

Do lat 80. większość systemów NLP wykorzystywała złożone, „odręczne” reguły. Jednak pod koniec lat 80. nastąpiła rewolucja w NLP. Było to wynikiem zarówno stałego wzrostu mocy obliczeniowej, jak i przejścia na algorytmy uczenia maszynowego. Podczas gdy niektóre z wczesnych algorytmów uczenia maszynowego (drzewa decyzyjne stanowią dobry przykład) stworzyły systemy podobne do odręcznych reguł starej szkoły, badania w coraz większym stopniu koncentrowały się na modelach statystycznych.

W latach 80-tych IBM był odpowiedzialny za opracowanie kilku udanych, skomplikowanych **modeli statystycznych**.

Elementy historii

W latach 90. popularność **modeli statystycznych** do analizy procesów języka naturalnego gwałtownie wzrosła. Czysto statystyczne metody NLP stały się niezwykle cenne w nadążaniu za ogromnym przepływem tekstu online. N-Gramy stały się użyteczne, rozpoznając i śledząc liczbowo skupiska danych językowych.

W 1997 roku wprowadzono modele rekurencyjnych sieci neuronowych LSTM (RNN), które w 2007 roku znalazły zastosowanie w przetwarzaniu głosu i tekstu.

Obecnie modele sieci neuronowych (LLM) są uważane za wiodące w badaniach i rozwoju w zakresie rozumienia tekstu i generowania mowy przez NLP.

N-gram – model językowy stosowany w rozpoznawaniu mowy

Zastosowanie n-gramów wymaga zgromadzenia odpowiednio dużego zasobu danych statystycznych – korpusu. Utworzenie modelu n-gramowego zaczyna się od zliczania wystąpień sekwencji o ustalonej długości n w istniejących zasobach językowych. Zwykle analizuje się całe teksty i zlicza wszystkie pojedyncze wystąpienia (1-gramy, unigramy), dwójki (2-gramy, bigramy) i trójki (3-gramy, trigramy). Aby uzyskać 4-gramy słów potrzebnych jest bardzo dużo danych językowych, co szczególnie dla języka polskiego jest trudne do zrealizowania.

Google Ngram Viewer lub Google Books Ngram Viewer to wyszukiwarka internetowa, która wykreśla częstotliwości dowolnego zestawu wyszukiwanych ciągów znaków na podstawie rocznej liczby n-gramów

Stan obecny

Rok **2023** był rokiem niezwykłego postępu w przetwarzaniu języka naturalnego (NLP) ze względu na udostępnienie **GPT-3**.

Nastąpiła niezwykła ewolucja dużych modeli językowych (**LLM**) dzięki inicjatywom open source oraz dynamicznej interakcji technologii i etyki w rozwoju sztucznej inteligencji.

Stan obecny

Chatbot Arena

<https://lmarena.ai/?leaderboard>

Rank ↕	Rank Spread ⓘ	Model ↕
1	1 - 6	 claude-opus-4-7-thinking Anthropic · Proprietary
2	1 - 5	 claude-opus-4-6-thinking Anthropic · Proprietary
3	1 - 8	 claude-opus-4-7 Anthropic · Proprietary
4	1 - 6	 claude-opus-4-6 Anthropic · Proprietary
5	1 - 9	 muse-spark Meta · Proprietary
6	2 - 9	 gemini-3.1-pro-preview Google · Proprietary

HELM

<https://crfm.stanford.edu/helm/lite/latest/#/leaderboard>

GPT-4o (2024-05-13)	0.938 
GPT-4o (2024-08-06)	0.928 
DeepSeek v3	0.908 
Claude 3.5 Sonnet (20240620)	0.885 
Amazon Nova Pro	0.885 
GPT-4 (0613)	0.867 
	0.864 
Llama 3.1 Instruct Turbo (405B)	0.854 

Open LLM

https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard

Stan z 2025 roku

Rank* (UB) ▲	Rank (StyleCtrl)	Model
1	1	Gemini-2.5-Pro-Exp-03-25
2	1	o3-2025-04-16
2	3	ChatGPT-4o-latest (2025-03-26)
3	5	Grok-3-Preview-02-24
3	5	Gemini-2.5-Flash- Preview-04-17

Model ^	Mean win rate
GPT-4o (2024-05-13)	0.938 ↗
GPT-4o (2024-08-06)	0.928 ↗
DeepSeek v3	0.908 ↗
Claude 3.5 Sonnet (20240620)	0.885 ↗

TEXT PROMPTS

Pareto Frontier

Model performance at each price point



Pareto Frontier Models

Top performers for their cost

AI	claude-opus-4-6-thinki...	1503
	Anthropic · Proprietary	\$20/M
G	gemini-3.1-pro-preview	1492
	Google · Proprietary	\$9.50/M
XI	grok-4.20-beta-0309-re...	1480
	xAI · Proprietary	\$5/M
G	gemini-3-flash	1474
	Google · Proprietary	\$2.38/M
G	gemma-4-31b	1451
	Google · Apache 2.0	\$0.34/M
S	step-3.5-flash	1392
	StepFun · Apache 2.0	\$0.25/M
MI	mimo-v2-flash (non-t...	1392
	Xiaomi · MIT	\$0.24/M
G	gemma-3-27b-it	1366
	Google · Gemma	\$0.14/M
G	gemma-3-12b-it	1342
	Google · Gemma	\$0.11/M
G	gemma-3n-e4b-it	1318
	Google · Gemma	\$0.10/M
G	gemma-3-4b-it	1303
	Google · Gemma	\$0.07/M
L	llama-3-8b-instruct	1223
	Meta · llama 3 Community	\$0.04/M



Hugging Face

https://huggingface.co/models

Main

Tasks

Libraries

Languages

Licenses

Other

Tasks

Text Generation

Any-to-Any

Image-Text-to-Text

Image-to-Text

Image-to-Image

Text-to-Image

Text-to-Video

Text-to-Speech

+ 44

Parameters

< 1B

6B

12B

32B

128B

> 500B

Libraries

PyTorch

TensorFlow

JAX

Transformers

Diffusers

sentence-transformers

Safetensors

ONNX

GGUF

Transformers.js

MLX

+ 42

Apps

vLLM

llama.cpp

MLX LM

LM Studio

Ollama

Jan

Draw Things

+ 7

Inference Providers

Groq

Novita

Cerebras

SambaNova

Nscale

fal

Hyperbolic

Together AI

+ 10

Models 2,801,035

Filter by name

Full-text search

Inference Available

Sort: Trending

Qwen/Qwen3.6-35B-A3B

Image-Text-to-Text · 36B · Updated 4 days ago · 209k · 890

MiniMaxAI/MiniMax-M2.7

Text Generation · 229B · Updated 2 days ago · 289k · 973

baidu/ERNIE-Image

Text-to-Image · Updated 3 days ago · 3.76k · 461

Jiunsong/supergemma4-26b-uncensored-gguf-v2

Text Generation · 25B · Updated 7 days ago · 72.5k · 415

baidu/ERNIE-Image-Turbo

Text-to-Image · Updated 3 days ago · 4.76k · 316

google/gemma-4-31B-it

Image-Text-to-Text · 33B · Updated 9 days ago · 4M · 2.17k

dealignai/Gemma-4-31B-JANG_4M-CRACK

Image-Text-to-Text · 6B · Updated 3 days ago · 160k · 1.29k

nvidia/Lyra-2.0

Updated 1 day ago · 132 · 211

unsloth/ERNIE-Image-Turbo-GGUF

Text-to-Image · 8B · Updated 5 days ago · 26.4k · 168

OpenMOSS-Team/MOSS-TTS-Nano-100M

tencent/HY-Embodied-0.5

Image-Text-to-Text · 4B · Updated 5 days ago · 1.6k · 869

unsloth/Qwen3.6-35B-A3B-GGUF

Image-Text-to-Text · 35B · Updated 3 days ago · 662k · 489

tencent/HY-World-2.0

Image-to-3D · Updated 3 days ago · 453

zai-org/GLM-5.1

Text Generation · 754B · Updated 3 days ago · 113k · 1.41k

OBLITERATUS/gemma-4-E4B-it-OBLITERATED

Text Generation · 8B · Updated about 11 hours ago · 37.1k · 329

openbmb/VoxCPM2

Text-to-Speech · Updated 3 days ago · 51.6k · 1.13k

HauhauCS/Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggressive

Image-Text-to-Text · 35B · Updated 3 days ago · 173k · 225

NucleusAI/Nucleus-Image

Text-to-Image · Updated 4 days ago · 1.18k · 180

Comfy-Org/ERNIE-Image

Updated 5 days ago · 161

Qwen/Qwen3.6-35B-A3B-FP8

Worek słów - Bag of Words

Problem z modelowaniem tekstu polega na tym, że jest on chaotyczny, natomiast techniki takie jak algorytmy uczenia maszynowego preferują dobrze zdefiniowane dane wejściowe i wyjściowe o stałej długości.

Algorytmy uczenia maszynowego nie mogą bezpośrednio pracować z surowym tekstem; **tekst należy przekonwertować na liczby.**

W szczególności na wektory liczb.

Nazywa się to ekstrakcją lub kodowaniem cech.

Popularną i prostą metodą ekstrakcji cech z danych tekstowych jest tzw. model tekstu typu bag-of-words.

Praca z tekstem

1. Przygotowanie danych tekstowych
 - a. Czyszczenie tekstu przy użyciu biblioteki NLTK, Keras lub Torchtext
 - b. Tokenizacja tekstu
2. Model języka Bag-of-Words
 - a. Przygotowanie słownika metodami: CountVectorizer, TfidfVectorizer, HashingVectorizer
 - b. Zarządzanie słownikiem: punktacja, ograniczanie, archiwizacja

```
torchtext.data.utils.get_tokenizer(tokenizer, Language='en') \[SOURCE\]
```

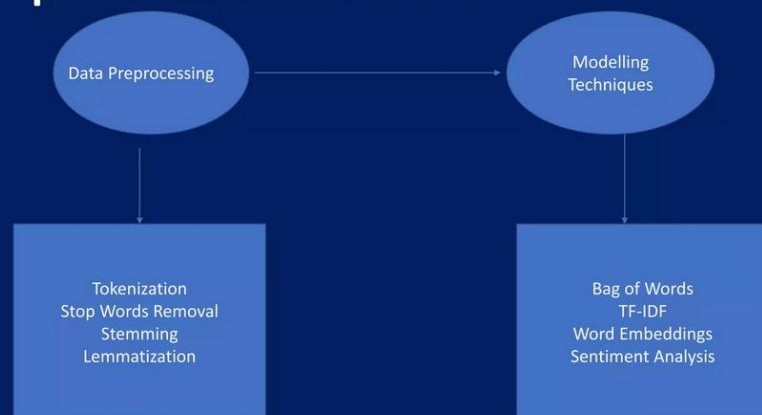
4. Word Embeddings-osadzenie słów

- a. Osadzanie słów – algorytmy Continuous-bag-of-words; Skip-gram
- b. Word2Vec Embedding i Stanford's GloVe Embedding

Zestaw narzędzi **Natural Language Toolkit**, w skrócie NLTK, to biblioteka języka Python napisana do pracy i modelowania tekstu. Zapewnia dobre narzędzia do ładowania i czyszczenia tekstu, których możemy użyć, aby przygotować nasze dane do pracy z algorytmami uczenia maszynowego i głębokiego uczenia się.

Czyszczenie tekstu

Steps towards NLP



This is a sample
↓
Tokenization
↓
[This] [is] [a] [sample]

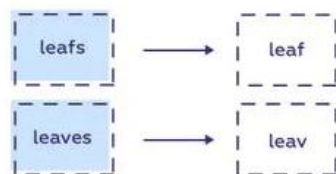
<https://medium.com/data-science-in-your-pocket/tokenization-algorithms-in-natural-language-processing-nlp-1fceb8454af>

Stop Words Example

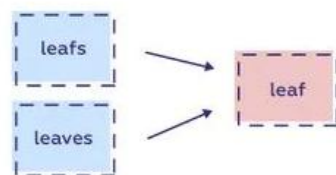
Sample Text with Stop Words	Sample Text without Stop Words
Aarush Coaching Classes – A stem learning place for kids	Aarush Coaching Classes, Stem, Learning, Place, kids
Can Listening be exhausting ?	Listening, Exhausting
I like Teaching, so I teach	Like, Teaching, Teach

<https://www.slideshare.net/slideshow/nlp-pptpptx/252713630>

Stemming



Lemmatization



sciforce



Resource: <https://blog.aaroncswong.com/2019/building-a-bigram-hidden-markov-model-for-part-of-speech-tagging/>

Packages supporting Polish language text clearing

Sentence analysis (tokenization, lemmatization, POS tagging etc.)

- SpaCy - A popular library for NLP in Python which includes Polish models for sentence analysis.
- Stanza - A collection of neural NLP models for many languages from Stanford NLP.
- Trankit - A light-weight transformer-based python toolkit for multilingual natural language processing by the University of Oregon.
- KRNNT and KFTT - Neural morphosyntactic taggers for Polish.
- Morfeusz - A classic Polish morphosyntactic tagger.
- Language Tool - Java-based open source proofreading software for many languages with sentence analysis tools included.
- Stempel - Algorithmic stemmer for Polish.
- PoLemma - pT5-based lemmatizer of named entities and multi-word expressions for Polish, available in small, base and large sizes.

Korpusy

O korpusie

Korpus to dowolny zbiór tekstów, w którym czegoś szukamy...

Korpus tekstów musi być odpowiednio zrównoważony gatunkowo, chronologicznie, stylowo, terytorialnie i pod innymi względami, np. ze względu na wiek i płeć autorów. Rodzaj zrównoważenia korpusu zależy od celów, jakim korpus służy.

Nasz korpus tekstów polskich to fragment słownikowej kuchni, czyli autentyczny materiał językowy, na którego podstawie opisujemy znaczenia słów i konstrukcji. Pojedyncze zdania z tekstów korpusu zamieszczamy w słownikach jako przykłady ilustrujące znaczenia.

Korpus wykorzystywany do tworzenia słowników ogólnych powinien gromadzić teksty z różnych dziedzin tematycznych, stylów i źródeł.

Zrównoważony tematycznie i gatunkowo **Korpus Języka Polskiego PWN** liczy **70 milionów słów**. Cały korpus, włączając archiwa prasowe i klasykę literacką od średniowiecza, **zawiera 100 milionów słów**. Składa się z tekstów książek, czasopism, druków ulotnych i akcydensowych (np. reklam, instrukcji obsługi, regulaminów, ulotek wyborczych), stron internetowych oraz tekstów mówionych. W porównaniu z innymi korpusami na świecie nasz zbiór zawiera dość dużo tekstów literackich. Postanowiliśmy bowiem uwzględnić szczególnie żywą w Polsce tradycję autorytetu kulturalnego jako kryterium poprawności językowej.

WebText

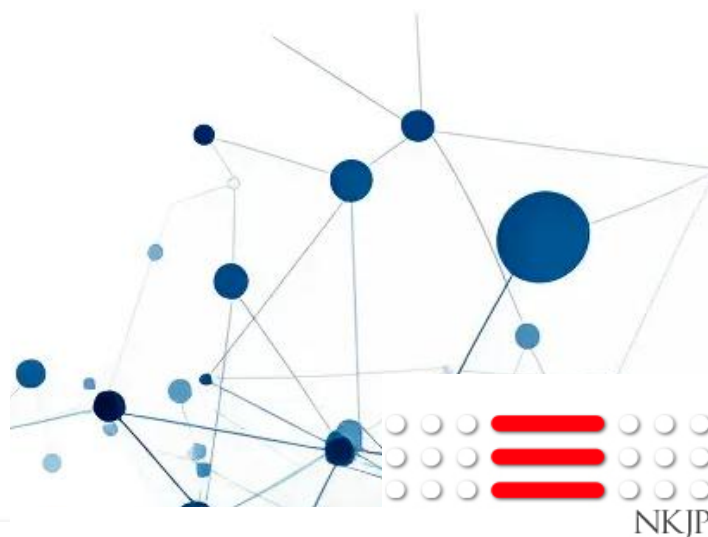
Introduced by Radford et al. in [Language Models are Unsupervised Multitask Learners](#)

WebText is an internal OpenAI corpus created by scraping web pages with emphasis on document quality. The authors scraped all outbound links from Reddit which received at least 3 karma. The authors used the approach as a heuristic indicator for whether other users found the link interesting, educational, or just funny.



OpenWebText

Introduced by Aaron Gokaslan et al. in [OpenWebText corpus](#)



Hugging Face

NATIONAL CORPUS OF
POLISH

Worek słów - Bag of Words

Nazwa worek słów związana jest z faktem, że wszelkie informacje o kolejności lub strukturze słów w dokumencie są odrzucane. Model uwzględnia jedynie to, czy dane słowa występują w dokumencie, a nie gdzie są w dokumencie. Brak uwzględnienia kontekstu.

Bag – of – Words model

Krok 1: Zbierz dane

Poniżej znajduje się fragment pierwszych kilku linijek tekstu z książki A Tale of Two Cities autorstwa Charlesa Dickensa, zaczerpnięte z Projektu Gutenberg.

```
It was the best of times,  
it was the worst of times,  
it was the age of wisdom,  
it was the age of foolishness,
```

Przykładowy tekst z Opowieść o dwóch miastach przez Charlesa Dickensa.

W tym małym przykładzie potraktujemy każdą linię jako osobny document a 4 linie to cały nasz zbiór dokumentów.

Krok 2: Zaprojektuj słownictwo

Teraz możemy stworzyć listę wszystkich słów naszego modelowego słownictwa. Unikalne słowa (ignorując wielkość liter i interpunkcję) to:

```
it  
was  
the  
best  
of  
times  
worst  
age  
wisdom  
foolishness
```

Lista unikalnych słów.

Jest to słownictwo składające się z 10 słów pochodzące z korpusu (dokumentu) zawierającego 24 słowa.

Krok 3: Utwórz wektory dokumentu

Następnym krokiem jest punktowanie słów w każdym dokumencie. Celem jest przekształcenie każdego dokumentu zawierającego dowolny tekst w wektor, którego możemy użyć jako danych wejściowych lub wyjściowych w modelu uczenia maszynowego. Ponieważ wiemy, że słownictwo składa się z 10 słów, możemy użyć reprezentacji 10 w dokumencie o stałej długości, z jedną pozycją w wektorze, aby ocenić każde słowo. Najprostszą metodą punktacji jest oznaczenie obecności słów jako wartości logicznej, 0 dla nieobecności, 1 dla obecności. Stosując dowolną kolejność słów wymienionych powyżej w naszym słowniku, możemy przejść przez pierwszy dokument i przekonwertować go na wektor binarny. Punktacja dokumentu będzie wyglądać następująco:

```
it = 1  
was = 1  
the = 1  
best = 1  
of = 1  
times = 1  
worst = 0  
age = 0  
wisdom = 0  
foolishness = 0
```

Lista unikalnych słów i ich występowania w dokumencie.

Jako wektor binarny wyglądałoby to następująco:

```
[1, 1, 1, 1, 1, 1, 0, 0, 0, 0]
```

Listing 8.4: Pierwsza linia tekstu w postaci wektora binarnego.

- Usunąć znaki interpunkcyjne z każdego tokenu.
- Odfiltruj pozostałe tokeny, które nie są alfabetyczne.
- Odfiltruj tokeny będące słowami kończącymi

Punktacja słów (Scoring Words)

Po wybraniu słownictwa należy ocenić występowanie słów w przykładowych dokumentach. W opracowanym przykładzie widzieliśmy już jedno bardzo proste podejście do punktacji: binarną punktację obecności lub braku słów. Niektóre dodatkowe proste metody punktacji obejmują:

- **~Counts.** Policz, ile razy każde słowo pojawia się w dokumencie.
- **~Frequencies.** Oblicz, jak często każde słowo pojawia się w dokumencie

Haszowanie słów

Być może pamiętasz z informatyki, że funkcja hash odwzorowuje dane na zbiór liczb o stałym rozmiarze. Na przykład używamy ich w tabelach skrótów podczas programowania, gdzie na przykład nazwy są konwertowane na liczby w celu szybkiego wyszukiwania. Możemy użyć hashu reprezentującego znane słowa w naszym słownictwie. Rozwiązuje to problem posiadania bardzo dużego słownictwa w przypadku dużego korpusu tekstowego, ponieważ możemy wybrać rozmiar przestrzeni mieszającej, która z kolei jest rozmiarem wektorowej reprezentacji dokumentu.

Słowa są hashed deterministycznie do tego samego indeksu całkowitego w docelowej przestrzeni. Następnie do oceny słowa można zastosować wynik lub liczbę binarną. Nazywa się to hash trick or feature hashing. Trudność polega na wybraniu przestrzeni hash, aby pomieścić wybrany rozmiar słownictwa i aby zminimalizować prawdopodobieństwo kolizji i rzadkości kompromisu.

TF-IDF

Problem z częstotliwością punktowania słów polega na tym, że w dokumencie zaczynają dominować słowa bardzo częste (np. większa liczba punktów), ale mogą nie zawierać tak dużo treści informacyjnej dla modelu jako rzadsze, ale być może słowa specyficzne dla domeny. Jednym z podejść jest przeskalowanie częstotliwości występowania słów według częstotliwości ich występowania we wszystkich dokumentach, tak aby wyniki dla częstych słów, takich jak te które również często pojawiają się we wszystkich dokumentach, są karane. To podejście do scoringu nazywa się Częstotliwością Terminów - Odwrotną Częstotliwością Dokumentów, w skrócie TF-IDF, gdzie:

- **~Term Frequency:** jest ocena częstotliwości występowania słowa w bieżącym dokumencie.
- **~Inverse Document Frequency:** to ocena rzadkości występowania danego słowa w dokumentach.

Wyniki stanowią wagę, w przypadku której nie wszystkie słowa są równie ważne i interesujące. Punktacja ma na celu podkreślenie wyrazów, które w danym dokumencie wyróżniają się odrębnością (zawierają przydatne informacje).

Zatem idf rzadkiego terminu jest wysoki, podczas gdy idf częstego terminu jest raczej niski.

Przykłady Python

- **TfidfVectorizer** **TFIDF-term frequency** (podsumowuje, jak często dane słowo pojawia się w słowniku), inverse document frequency - zmniejsza skalę słów, które często pojawiają się w dokumentach.
- **Hashing**-konwertowanie tekstu na wektor o wybranym wymiarze; każdemu tokenowi przypisywana jest wartość (indeks)
- **CountVectorizer**-dzieli tekst na tokeny, buduje słownik wszystkich unikalnych tokenów w zbiorze, liczy wystąpienia tokenów w dokumencie i tworzy macierz wartości dla dokumentów i słów.

Rdzeń-pień-stem

Stemming odnosi się do procesu sprowadzania każdego słowa do rdzenia lub podstawy.

Na przykład fishing, fished, fisher wszystko sprowadza się do fish.

Niektóre aplikacje, takie jak klasyfikacja dokumentów, mogą skorzystać na stemmingu i redukcji słownika, ze względu na skupienie się na sensie lub nastroju dokumentu, a nie na głębszym znaczeniu.

Istnieje wiele algorytmów stemming, chociaż popularną i długoletnią metodą jest algorytm Porter Stemming. Ta metoda jest dostępna w NLTK poprzez klasę PorterStemmer class.

Metoda	Co zawiera wektor
CountVectorizer	surowe liczby wystąpień
TF-IDF	ważone częstości
HashingVectorizer	hashe zamiast słownika
Word2Vec	znaczenie semantyczne
BERT	kontekst i semantyka

Osadzanie słów (Word embedding)

- <https://code.google.com/archive/p/word2vec/>

Osadzanie słów to rodzaj reprezentacji słów, w którym słowa o podobnym znaczeniu mają podobną reprezentację.

Zazwyczaj reprezentacja jest wektorem o wartościach rzeczywistych, który koduje znaczenie słowa w taki sposób, że oczekuje się, że słowa znajdujące się bliżej w przestrzeni wektorowej będą miały podobne znaczenie.

Osadzanie słów (Word embedding)

Oprogramowanie do trenowania i używania osadzania słów obejmuje:

- Word2vec Tomáša Mikolova,
- GloVe Uniwersytetu Stanforda,
<https://wikipedia2vec.github.io/demo/>
- GN-GloVe,
- Flair embeddings,
- i inne

Znana biblioteka GENSIM



Radim Řehůřek: Creator of **Gensim** living in Prague.

What is Gensim?

Gensim is a free open-source Python library for representing documents as semantic vectors, as efficiently (computer-wise) and painlessly (human-wise) as possible.

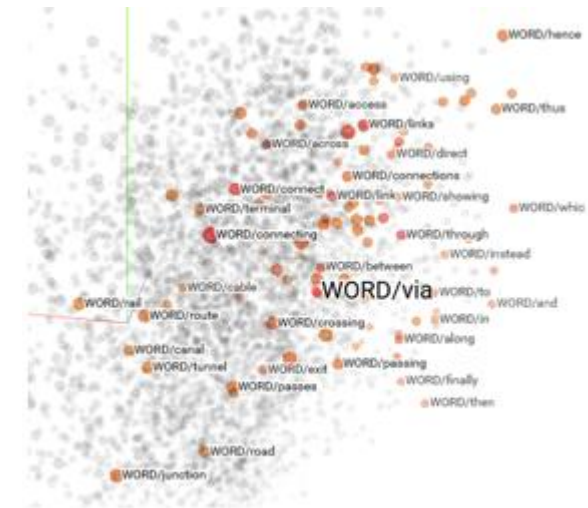
Metody osadzania słów

Metody osadzania słów uczą się reprezentacji wektorowej o wartości rzeczywistej dla słownictwa o określonym rozmiarze ze **słownictwa korpusu tekstu**.

Proces uczenia się jest albo połączony z modelem sieci neuronowej w ramach jakiegoś zadania, takiego jak klasyfikacja dokumentów, lub jest procesem nienadzorowanym, wykorzystującym statystyki dokumentów.

Embedding Layer- warstwa osadzania

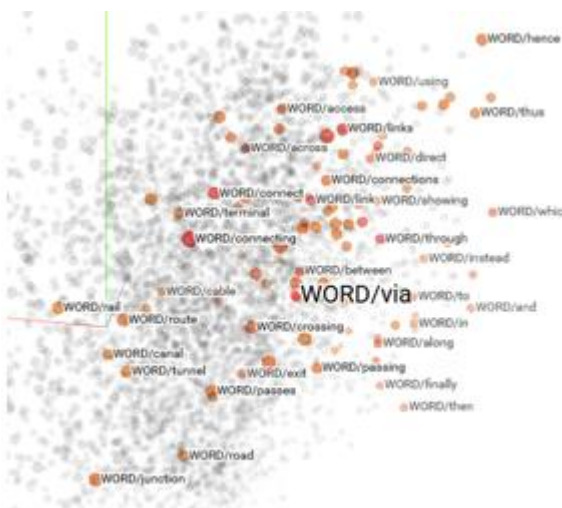
<https://wikipedia2vec.github.io/demo/>



Word2Vec

Word2Vec to metoda skutecznego uczenia się samodzielnego osadzania słów z korpusu tekstowego.

Została ona opracowana przez Tomasa Mikolova i in. w Google w 2013 r. jako odpowiedź na potrzebę uczynienia osadzania w oparciu o sieć neuronową i od tego czasu stała się standardem w opracowywaniu wstępnie wytrenowanego osadzania słów.



Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov

Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen

Google Inc., Mountain View, CA
kaichen@google.com

Greg Corrado

Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean

Google Inc., Mountain View, CA
jeff@google.com

Word2Vec

- Continuous Bag-of-Words lub model CBOW.
- Continuous Skip-Gram Model.

Model CBOW uczy się osadzania, przewidyując bieżące słowo na podstawie jego kontekstu.

Model ciągłego skip-gramu uczy się, przewidyując otaczające słowa na podstawie bieżącego słowa.

<https://www.tensorflow.org/text/tutorials/word2vec>

Bag of Word

Założmy, że w naszym słowniku znajdują się słowa: *Rome, Italy, Paris, France* oraz *parametry* określające *country*. Możemy użyć każdego z tych słów, aby utworzyć reprezentację, używając one-hot schematu wszystkich słów, jak pokazano dla *Rome* na rysunku 2-1.

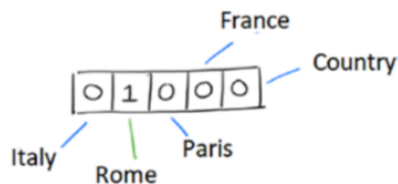


Figure 2-1. A representation of Rome

Korzystając z podejścia polegającego na przedstawieniu słów w postaci wektorowej, możemy niestety wykorzystać takie kodowanie tylko do testowania równości między słowami, porównując ich wektory.

Word Embeddings

Lepszą formę reprezentacji możemy uzyskać tworząc wiele hierarchii/wymiarów, w których informacjom związanym z każdym ze słów można przypisać różne wagi. Liczba tych wymiarów jest naszym wyborem, a każde ze słów będzie reprezentowane przez rozkład wag.

Teraz mamy nowy format reprezentacji słownej, przy użyciu różnych wymiarów dla każdego ze słów

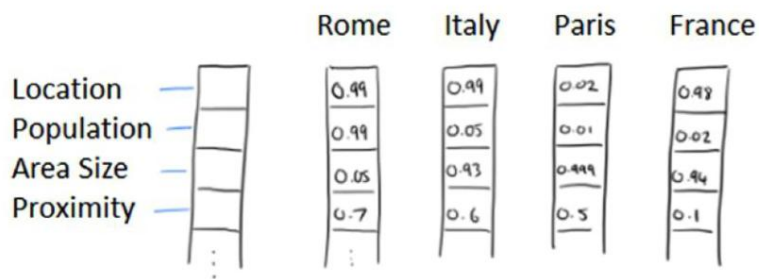
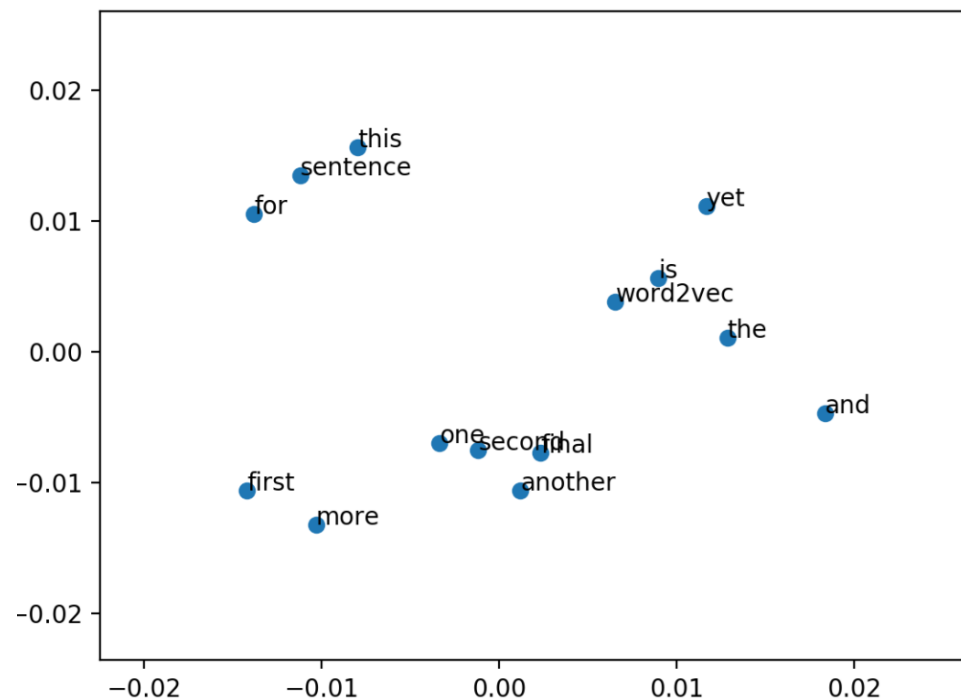


Figure Our representation



Wektory użyte dla każdego słowa oznaczają rzeczywiste znaczenie tego słowa i zapewniają lepszą skalę, za pomocą której można wykonać porównanie słów. Nowo utworzone wektory są wystarczające abyśmy byli w stanie odpowiedzieć na rodzaj relacji zachodzących między słowami.

Words Embedding

Model CBOW

Ciągły model bag-of-words ma podobną architekturę do FNNLM, jak pokazano na rysunku. Kolejność słów nie ma wpływu na warstwę projekcyjną i ważne jest, które słowa aktualnie wpadają do bag, aby przewidzieć słowo wyjściowe.

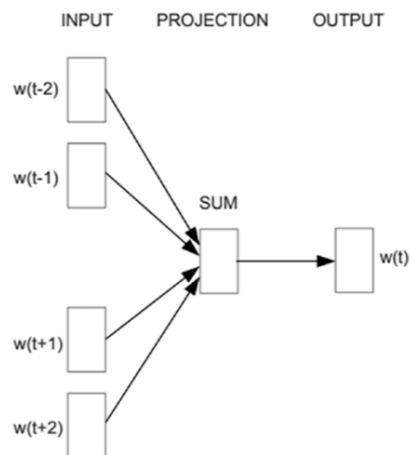


Figure Continuous bag-of-words model architecture

Model Skip-Gram

Model Skip-Gram modeluje otaczające słowa za pomocą bieżącego słowa w sekwencji. Wynik klasyfikacji otaczających słów uzyskuje się na podstawie relacji składniowej i wystąpień ze słowem środkowym. Każde słowo obecne w sekwencji jest traktowane jako dana wejściowa do klasyfikatora, w ten sposób algorytm przewiduje słowa należące do pewnego z góry określonego horyzontu słów występujących przed i po słowie środkowym.

Trzeba zdecydować się na kompromis między wyborem zakresu słów, a złożoność obliczeniowa i jakością wyniku. Gdy odległość do danego słowa wzrasta, odległe słowa są słabiej powiązane z bieżącym słowem, w porównaniu do bliższych słów. Ten problem rozwiązuje się, przypisując wagi jako funkcję odległości od środkowego słowa i nadając mniejsze wagi lub pobierając mniej słów do otoczenia słowa

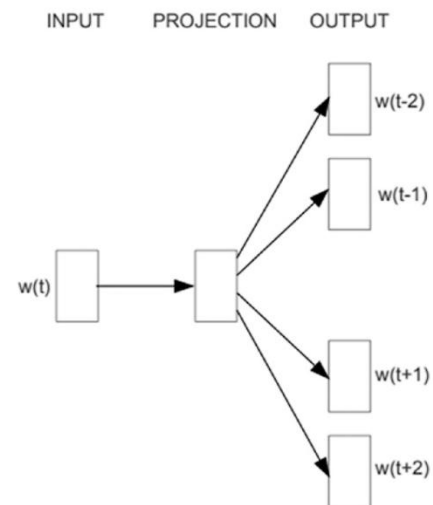


Figure 2-6. Skip-gram model architecture

Words Embedding

Na wektorach osadzeń można przeprowadzać działania:

```
queen = (king - man) + woman
```

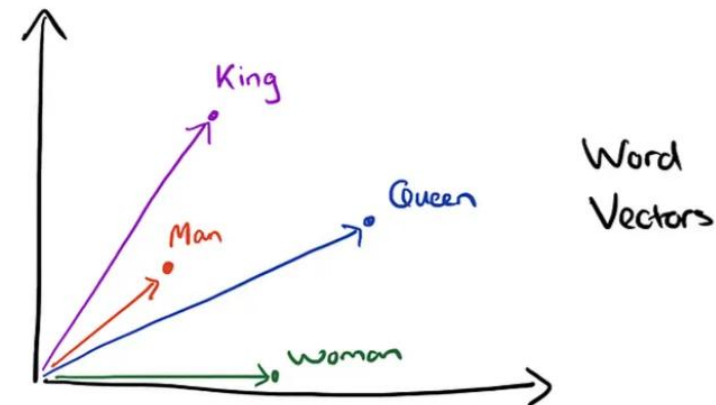
Oznacza to, że słowo królowa jest najbliższym słowem dla króla, jeżeli od pojęcia król odejmiemy pojęcie mężczyzna a dodamy kobieta.

```
import gensim.downloader as api
```

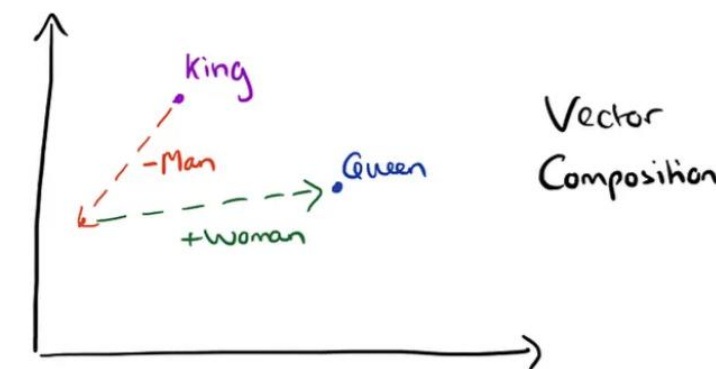
```
# Ładujemy GoogleNews-vectors-negative300 model wykorzystując gensim's downloader  
model = api.load("word2vec-google-news-300")
```

```
model.most_similar(  
    positive=['King', 'Woman'],  
    negative=['Man'],  
    topn=2  
)
```

```
[=====] 100.0% 1662.8/1662.8MB downloaded  
[('Queen', 0.4929387867450714), ('Tupou_V.', 0.45174285769462585)]
```



example of word vectors and vector composition. Source: Adrian Colyer.



Komponenty modelu: Ukryta warstwa

Sieć neuronowa budowana jest z jedną warstwą ukrytą, gdzie liczbę neuronów wybiera się jako równą liczbie cech lub wymiarów, według których chcemy reprezentować słowa osadzone. Na poniższym wykresie mamy warstwę ukrytą z macierzą wag zawierającą 300 kolumn, równa liczbie neuronów — co będzie liczbą cech w wektorze wyjściowym osadzania słów — liczba wierszy to 100 000, czyli równa wielkości słownika użytego do trenowania modelu.

Liczba neuronów jest uważana za hiperparametr modelu i można go zmienić w razie potrzeby. Model wytrenowany przez Google wykorzystuje 300 wymiarowe wektory cech i może to być dobry początek dla tych, którzy nie chcą trenować swojego zestawu modelu osadzania słów. Można użyć poniższego linku do pobrania wytrenowany zestaw wektorów: <https://code.google.com/archive/p/word2vec>.

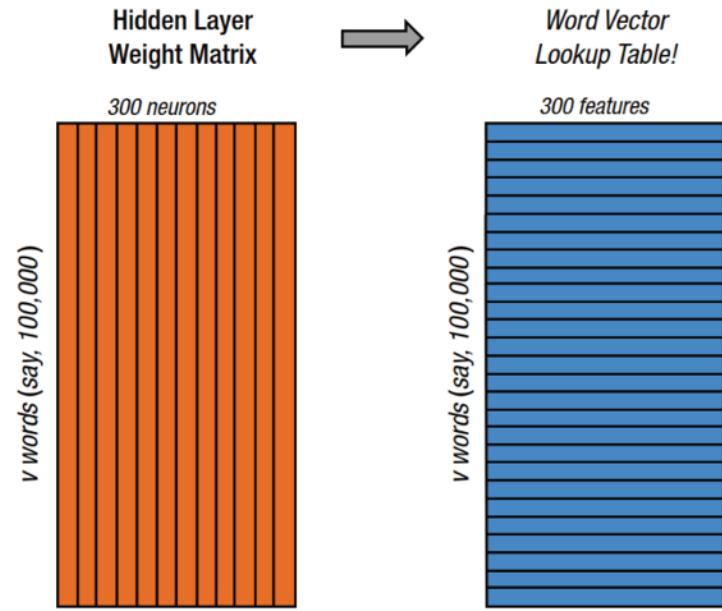


Figure 1. Weight matrix of the hidden layer and vector lookup table

Osadzanie Word2Vec

Word2Vec to jeden z algorytmów uczenia się do osadzania słów z korpusu tekstowego. Nie będziemy zagłębiać się w algorytmy poza tym, że ogólnie rzecz biorąc, przetwarzają one okno słów dla każdego słowa docelowego, aby zapewnić kontekst, a co za tym idzie, znaczenie słów. Podejście to opracował Tomas Mikolov, wcześniej pracujący w Google, a obecnie w Facebooku.

Osadzony słownik Stanford's GloVe

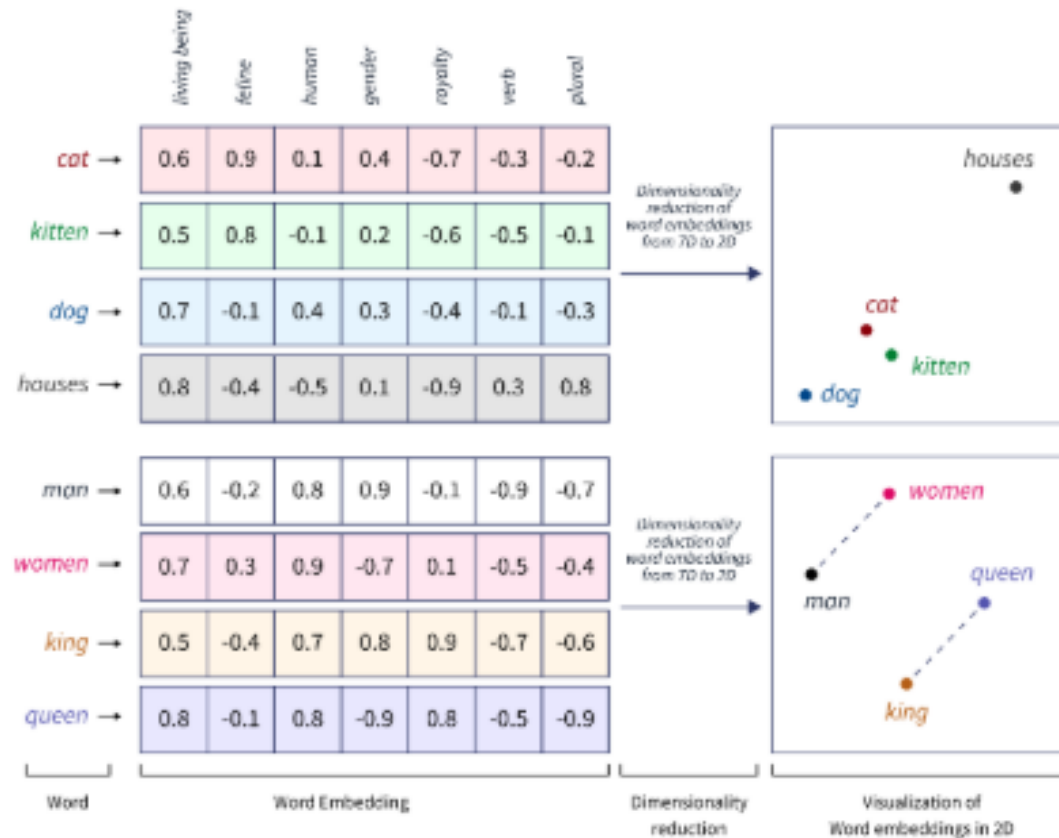
Stanford ma również swój własny algorytm osadzania słów, zwany Global Vectors for Word Representation lub w skrócie GloVe. Nie będę tutaj wdawał się w szczegóły różnic pomiędzy Word2Vec i GloVe, ale ogólnie rzecz biorąc, praktykujący NLP biorąc pod uwagę wyniki wydają się obecnie preferować GloVe.

Podobnie jak Word2Vec, GloVe również udostępnia wstępnie wytrenowane wektory słów, (duży wybór). Możesz pobrać wstępnie przeszkolone wektory słów GloVe i załadować je

Osadzanie słów

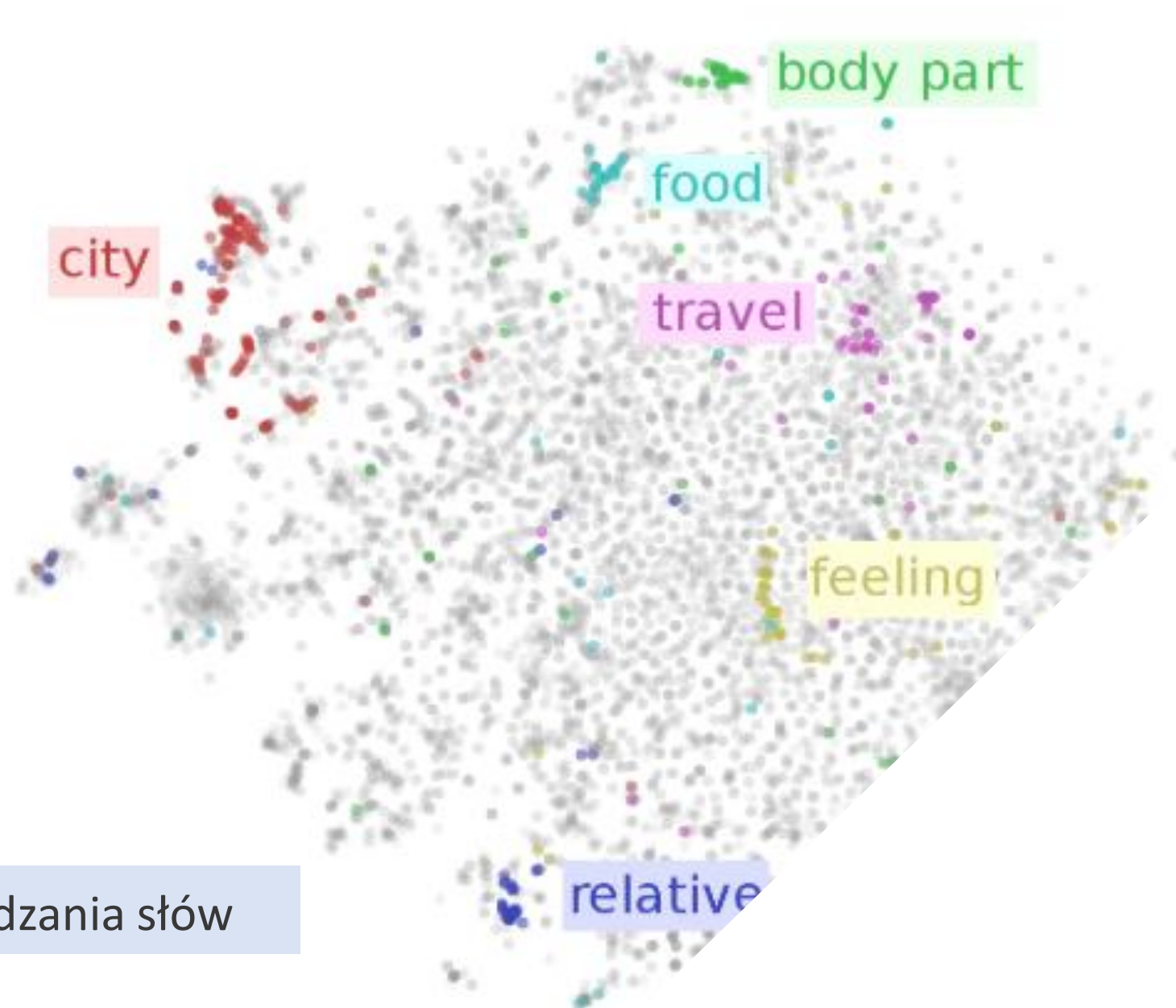
Dane przygotowane jako tokeny (pnie, chunks).

Do tokenów stosujemy osadzanie słów (word embedding)- tokeny reprezentowane są jako wielowymiarowe wektory.



Osadzanie słów to język AI. Każde słowo jest reprezentowane jako wielowymiarowy wektor, który zawiera jego znaczenie semantyczne w oparciu o kontekst w danych uczących. Te wektory pozwalają AI zrozumieć relacje i podobieństwa między słowami, zwiększając zrozumienie i wydajność modelu.

Wizualizacje osadzania słów (PCA, tSNE)



Algorytmy word2vec i GloVe osadzania słów

Zastosowanie

1. Analiza sentymentów

- a. Przygotowanie danych tekstowych
- b. Modele prognostyczne Sentiment Analysis
- c. Klasyfikacja tekstu
- d. Porównywanie i ocena modeli

2. Generowanie tekstu

- a. Modele sieci neuronowych do generowania dokumentów
- b. Statystyczne modelowanie warstwy semantycznej języka
- c. Opracowanie modelu Embedding + LSTM do generowania tekstu

Przykład

Plik Excel W7 NLP



Architektura Encoder-decoder

Encoder-dekoder (koder-dekoder) to rodzaj architektury sieci neuronowej, która jest używana do uczenia sekwencyjnego. Składa się ona z dwóch części, kodera i dekodera. Koder przetwarza sekwencję wejściową w celu utworzenia zestawu wektorów kontekstowych, które są następnie wykorzystywane przez dekodera do generowania sekwencji wyjściowej. Architektura ta umożliwia między innymi zadania takie jak tłumaczenie maszynowe, streszczanie tekstu, napisy do obrazów. Chodzi o to, aby móc przyjąć jedną formę danych (taką jak tekst) i przekonwertować ją na inną (taką jak obrazy). W ten sposób maszyny mogą nauczyć się rozumieć złożone relacje między różnymi typami danych i wykorzystywać je do bardziej wydajnego przetwarzania.

Architektura Encoder-decoder seq2seq

W podstawowym modelu seq2seq bez uwagi/attention, wektor kontekstu jest zazwyczaj końcowym stanem ukrytym generowanym przez koder (h).

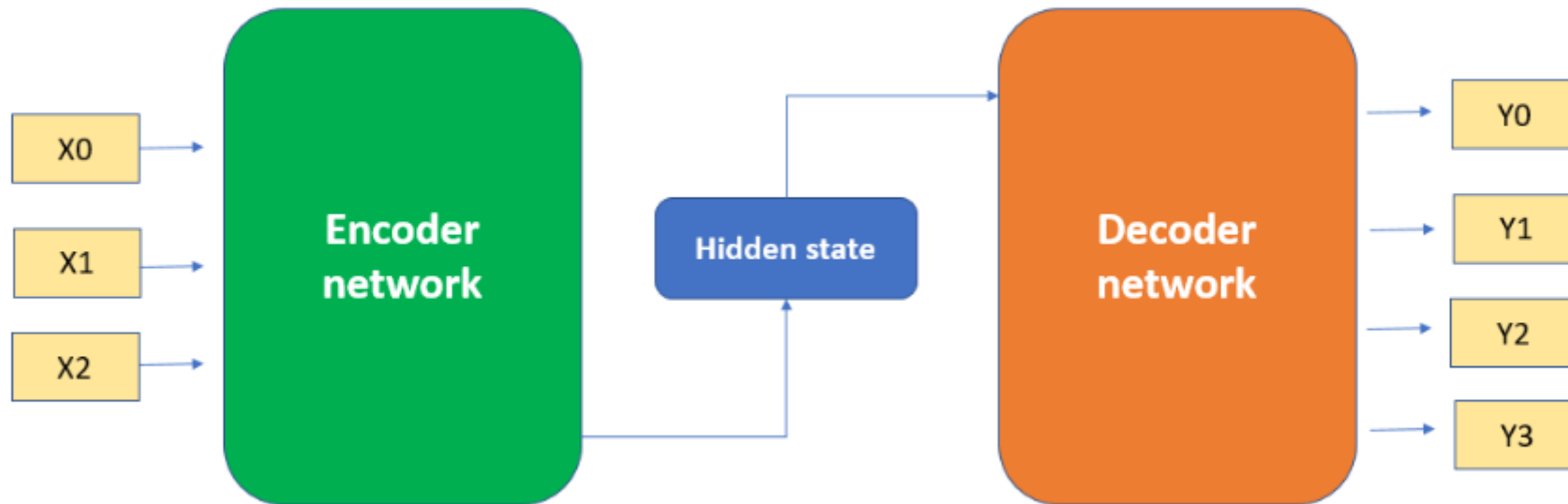
Ten stan ukryty jest następnie używany jako początkowy stan ukryty dla dekodera, który generuje sekwencję wyjściową krok po kroku.

Podczas dekodowania wektor kontekstu jest używany przez dekodera do generowania każdego elementu sekwencji wyjściowej. W każdym kroku dekodera pobiera poprzedni element wyjściowy i bieżący stan ukryty, a następnie tworzy nowy stan ukryty i element wyjściowy

Architektura koder-dekoder z sieciami neuronowymi

- Możemy wykorzystać CNN, RNN i LSTM w architekturze koder-dekoder do rozwiązywania różnego rodzaju problemów. Wykorzystanie kombinacji różnych typów sieci może pomóc w uchwyceniu złożonych relacji pomiędzy sekwencją danych wejściowych i wyjściowych.
- CNN jako koder, RNN/LSTM jako dekodek: Ta architektura może być używana do zadań takich jak napisy do obrazów, gdzie wejściem jest obraz, a wyjściem jest sekwencja słów opisujących obraz. CNN może wyodrębnić cechy z obrazu, podczas gdy RNN/LSTM może wygenerować odpowiednią sekwencję tekstową. CNN są dobre w wyodrębnianiu cech z obrazów i dlatego mogą być używane jako koder w zadaniach związanych z obrazami.
- RNN/LSTM są dobre w przetwarzaniu danych sekwencyjnych, takich jak sekwencje słów, i mogą być używane jako kodery w zadaniach związanych z obrazami.

Przykład Excel W10 NLP



- <https://nmtorch.org/text/main/datasets.html>

Datasets

- Text Classification

- AG_NEWS
- AmazonReviewFull
- AmazonReviewPolarity
- CoLA
- DBpedia
- IMDB
- MNLI
- MRPC
- QNLI
- QQP
- RTE
- SogouNews
- SST2
- STSB
- WNLI

`torchtext.datasets`

- + Text Classification
- + Language Modeling
- + Machine Translation
- + Sequence Tagging
- + Question Answer
- + Unsupervised Learning

- `torchtext.nn`

- MultiheadAttentionContainer
- InProjContainer
- ScaledDotProduct

- `torchtext.data.functional`

- generate_sp_model
- load_sp_model
- sentencepiece_numericalizer
- sentencepiece_tokenizer
- custom_replace
- simple_space_split
- numericalize_tokens_from_iterator
- filter_wikipedia_xml
- to_map_style_dataset

- `torchtext.data.metrics`

- bleu_score

- `torchtext.data.utils`

- get_tokenizer
- ngrams_iterator

Nie jest wspierana od 2024 roku? Są problemy z instalacją.

Duże modele językowe

Duże modele językowe (LLM), to zaawansowane systemy sztucznej inteligencji zaprojektowane do rozumienia i generowania tekstu podobnego do ludzkiego na podstawie dostarczonych im danych wejściowych.

Modele te charakteryzują się ogromną skalą, z miliardami, a nawet bilionami parametrów, które pozwalają im uchwycić skomplikowane wzorce i niuanse w języku.

2017 r.

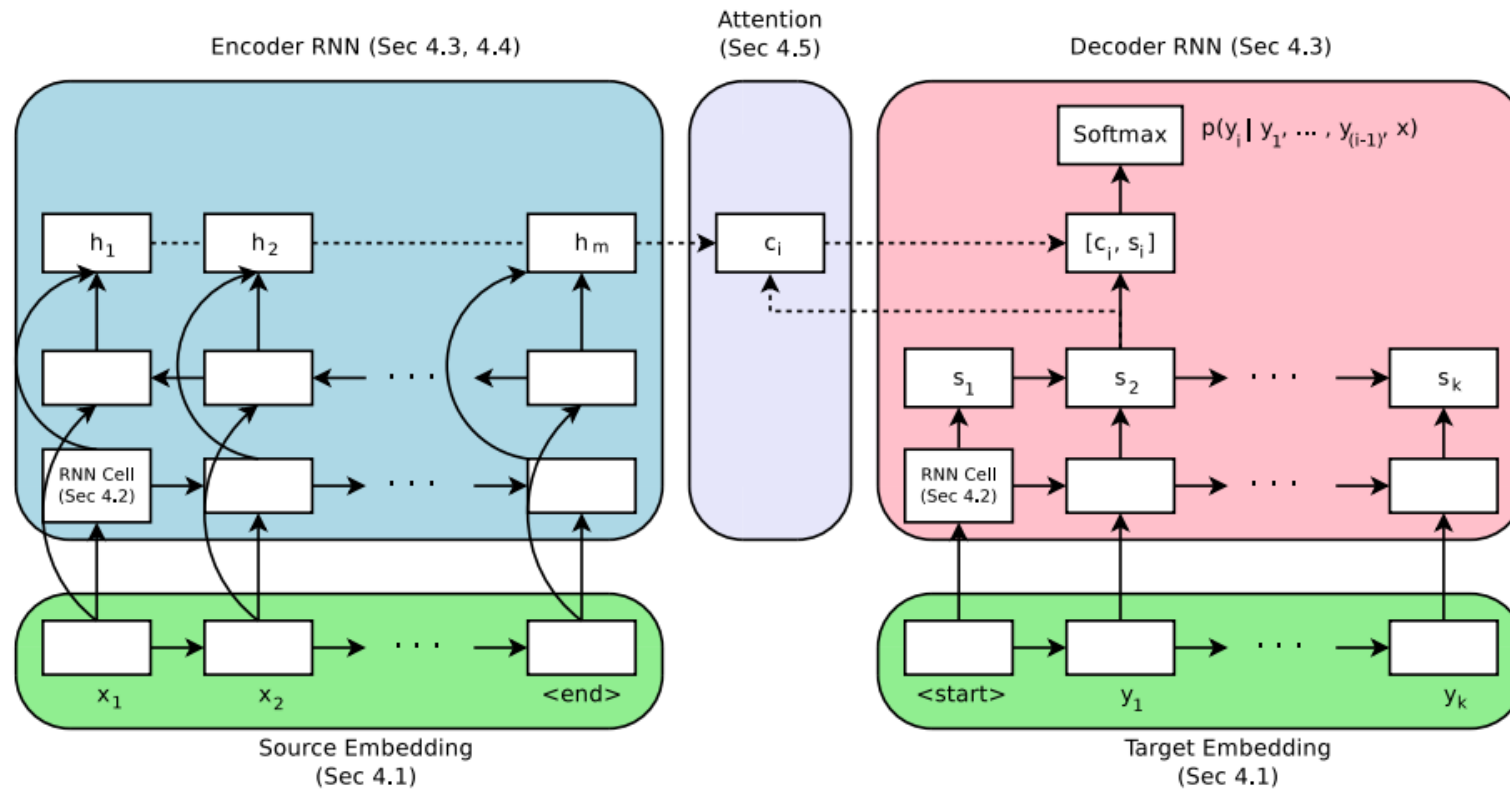


Figure 1: Encoder-Decoder architecture with attention module. Section numbers reference experiments corresponding to the components.

Massive Exploration of Neural Machine Translation Architectures

Denny Britz[†], Anna Goldie^{*}, Minh-Thang Luong, Quoc Le
{dennybritz, agoldie, thangluong, qvl}@google.com
Google Brain

Uwaga/Attention

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Biogram

Łukasz is the co-author of Tensorflow, the Tensor2Tensor and Trax libraries, and the Transformer paper. He is a Staff Research Scientist at Google Brain and his work has greatly influenced the AI community.

Attention is All You Need

A Vaswani · Cytowane przez 185976 –



arXiv

<https://arxiv.org> › cs · [Tłumaczenie strony](#) ⋮

[1706.03762] Attention Is All You Need

12 cze 2017 — We propose a new simple network architecture, the Transformer, based solely on **attention** mechanisms, dispensing with recurrence and convolutions entirely.



Institut Informatyki Uniwersytetu Wrocławskiego
<https://ii.uni.wroc.pl> › [Instytut](#) › [aktualności](#) ⋮

[Łukasz Kaiser - nasz absolwent - wśród autorów systemu ...](#)

18 mar 2023 — Jednym z core contributors systemu był nasz absolwent - Łukasz Kaiser, obecnie pracownik OpenAI, a wcześniej Google Brain. Łukasz pełnił rolę ...

„Japoński Nobel” dla Łukasza Kaisera

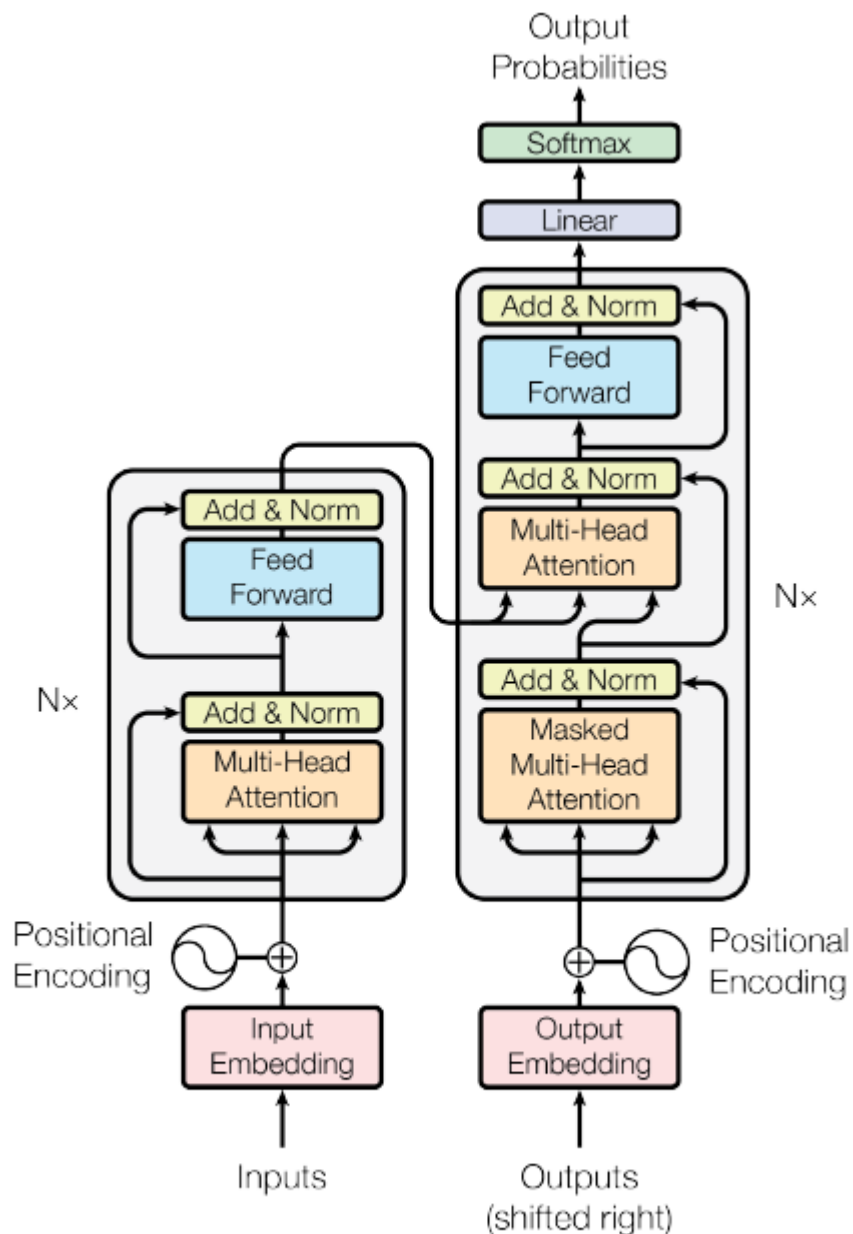
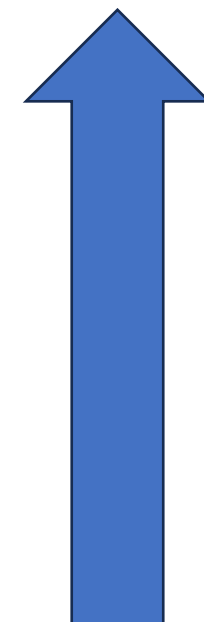


Figure 1: The Transformer - model architecture.

W pracy Attention is ..

Architektura koder- dekodek używana np. do tłumaczenia tekstu.

- Sieć gęsto połączona
- Połączenia resztowe
- Obliczanie samo-uwagi (self-Attention)
- Osadzenie słów i pozycji
- Osadzanie słów



Kodowanie pozycyjne lub pozycyjne osadzanie- wektor E

Zastępuje sekwencyjny wsad RNN (LSTM)

Kodowanie pozycyjne opiera się na wartościach funkcji sin i cos

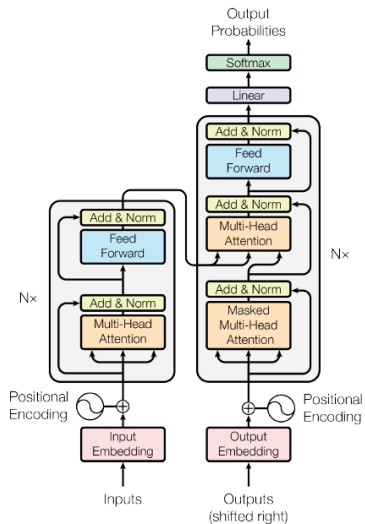
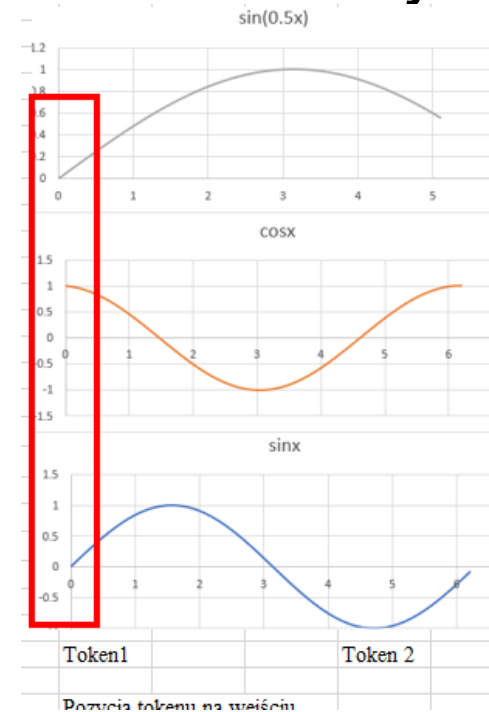


Figure 1: The Transformer - model architecture.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$



pozycyjne osadzanie-inna technika z pracy Gehring, Auli

1. Osadzanie słów (Word Embedding)

Macierz osadzania W_E : każdemu tokenowi odpowiada wektor o zadanym wymiarze.
GPT3 –wektor o 12 288 współrzędnych.

Macierz osadzania ma wymiar 12 288 wiersz i 50 257 kolumn.

Wagi macierzy osadzania W_E są wyznaczane w procesie uczenia.

```
CLASS torch.nn.Embedding(num_embeddings, embedding_dim, padding_idx=None,  
                        max_norm=None, norm_type=2.0, scale_grad_by_freq=False, sparse=False,  
                        _weight=None, _freeze=False, device=None, dtype=None) \[SOURCE\]
```

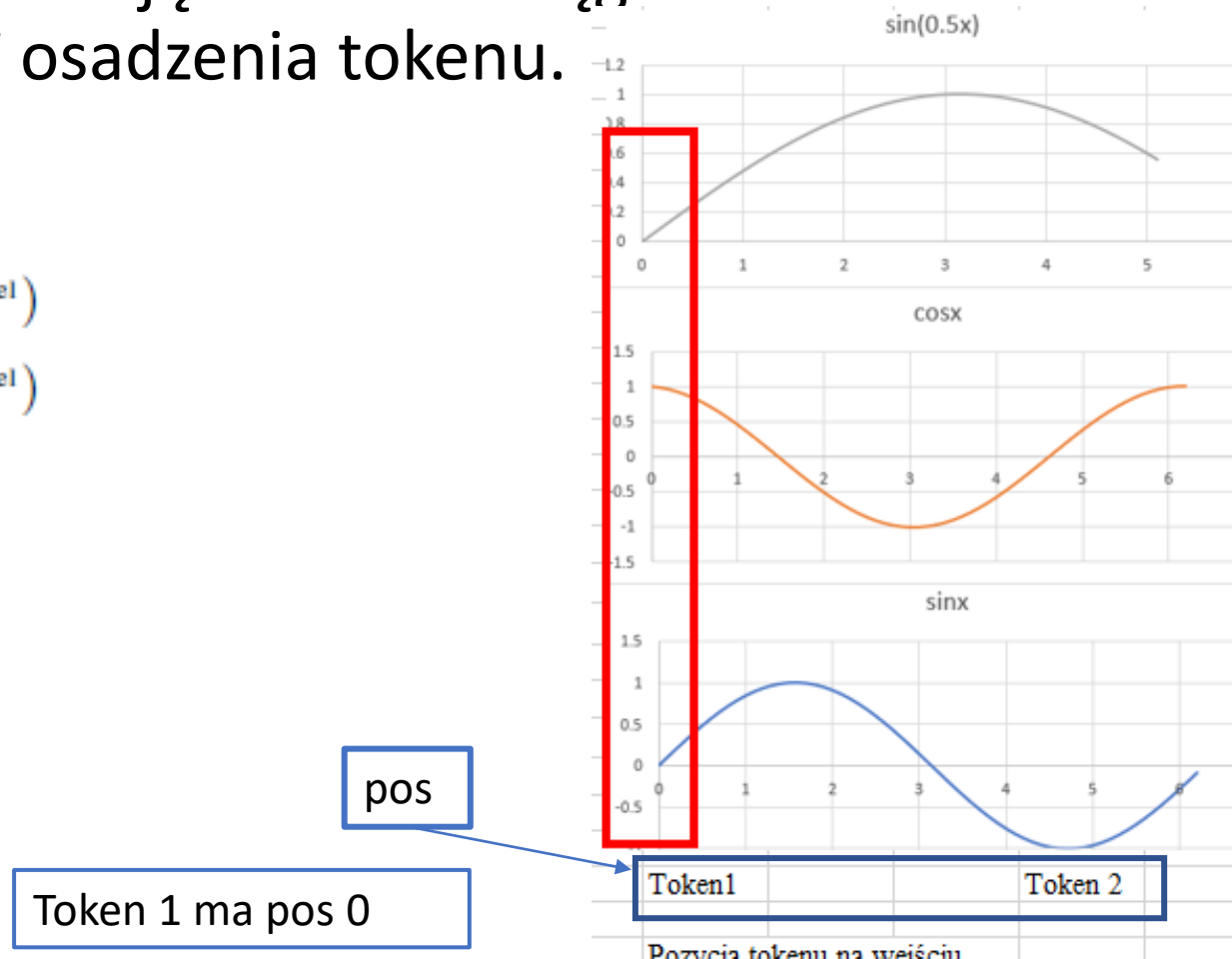
2. Kodowanie pozycyjne

Wartości wektora dla kodowania pozycyjnego mogą się powtarzać, ale daje on jednoznaczną reprezentację tokenu w ciągu tokenów o wymiarze równym wektorowi osadzenia tokenu.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

d_{model} = wymiar osadzania



3. Obliczanie uwagi

Komórka Samo-Uwagi (Self Attention)

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Iloczyn macierzy i osadzeń
poszczególnych tokenów

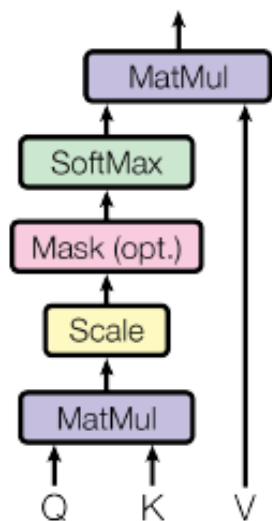
WQ		E2	Q
-0.8	-1.7	1.45	-2.367
0.4	2.8	0.71	2.568
Wk		E2	K
-1.5	0.7	1.45	-1.678
1.5	-2.1	0.71	0.684
Wk		E1	
-1.5	0.7	-2.38	4.34
1.5	-2.1	1.1	-5.88

Wv		E2	V2
1	-0.5	1.45	1.10
0.6	0.1	0.71	0.94
		E1	V1
1	-0.5	-2.38	-2.93
0.6	0.1	1.1	-1.318

Przykład w pliku Excel

Uwaga/samouwaga

Scaled Dot-Product Attention



- Q-query- zapytanie
- K- key klucz
- V-value wartość

Macierz parametrów W_Q

Macierz parametrów W_K

Macierz parametrów W_V

Wyznaczane w procesie uczenia, podobnie jak macierz osadzenia (embedding matrix)

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Obliczanie uwagi

Komórka Samo-Uwagi (Self Attention)

`torch.nn.functional.cosine_similarity`

`torch.nn.functional.cosine_similarity(x1, x2, dim=1, eps=1e-8) → Tensor`

$$\text{Attention}(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q-query- zapytanie

K- key -klucz

V-value wartość

d_k -wymiar -wektora zapytania

Kąt między wektorami

Ze wzoru na [iloczyn skalarny](#) można wyprowadzić wzór na cosinus kąta między wektorami.

$$\cos \alpha = \frac{\vec{a} \circ \vec{b}}{|\vec{a}| \cdot |\vec{b}|}$$

$\vec{a} \circ \vec{b}$ – iloczyn skalarny wektorów $\vec{a}$ i $\vec{b}$

$|\vec{a}|, |\vec{b}|$ – długość wektora $\vec{a}$ i $\vec{b}$

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Kąt między wektorami

Ze wzoru na **iloczyn skalarny** można wyprowadzić wzór na cosinus kąta między wektorami.

$$\cos \alpha = \frac{\vec{a} \circ \vec{b}}{|\vec{a}| \cdot |\vec{b}|}$$

$\vec{a} \circ \vec{b}$ – **iloczyn skalarny** wektorów $\vec{a}$ i $\vec{b}$

$|\vec{a}|, |\vec{b}|$ – **długość wektora** $\vec{a}$ i $\vec{b}$

Q ^T		K	
0.034	2.128	4.34	-12.3651
		-5.88	
		E1	Q
-1.7	-2.38	0.034	
2.8	1.1	2.128	
		K	
0.7	-2.38	4.34	
-2.1	1.1	-5.88	

Q ^T		K	
0.034	2.128	4.34	-12.3651
		-5.88	

W pliku Excel wektory osadzania są wektorami kolumnowymi. Stąd różnica w wzorze na iloczyn skalarny. Jest to mnożenie wiersza i kolumny.

Trochę o iloczynie skalarnym wektorów

- Przykład w materiałach

Tensor 1: `tensor([0.5000, 0.3000])`

Tensor 2: `tensor([0.3000, 0.2000])`

Tensor 3: `tensor([-0.3000, -0.5000])`

Tensor 4: `tensor([-0.2000, 0.5000])`

Cosine Similarity between Tensor 1 i Tensor 2 : `tensor(0.9989)`

Cosine Similarity between Tensor 1 i Tensor 3 : `tensor(-0.8824)`

Cosine Similarity between Tensor 1 i Tensor 4 : `tensor(0.1592)`

Self-Attention (samo-uwaga)

Terminy samo-uwaga i skalowana uwaga iloczynu skalarnego są używane zamiennie.

- Samo-uwaga mierzy jak bardzo słowo powinno skupiać się na innych słowach w zdaniu.

Aby to obliczyć wykonuje się następujące kroki.

1. Tworzy się 3 nowe wektory z każdego wektora słów wejściowych, zwane wektorami **zapytania, klucza i wartości**.

- Wektory te są oznaczone symbolami Q, K i V.
- Są one uzyskane przez pomnożenie wektora słów wejściowych przez trzy zestawy wag znajdujących podczas uczenia i tych samych dla całego promptu.

2. Używamy wektorów zapytań i kluczy do obliczenia wyniku. Wynik ten określa, ile uwagi należy zwrócić na inne słowa w zdaniu dla danego słowa.

Self-Attention (samo-uwaga)

3. Skalowanie wyniku przez stałą, tak aby jego wartość była pod kontrolą (stabilizuje szkolenie).
4. Normalizacja wyniku przy użyciu funkcji softmax, tak aby każdy wynik był dodatni, a suma wyników wszystkich słów w zdaniu wynosiła 1.
5. Pomnożenie wyniku przez wektor wartości, aby obliczyć wartość uwagi.

- Zapytanie reprezentuje bieżące słowo.
- Klucz reprezentuje wszystkie słowa w zdaniu.
- Wartość reprezentuje treść każdego słowa.

Wyznaczanie samouwagi



Samouwaga dla modelu GPT3 wyznaczana jest tylko dla tokenów poprzedzających

4. Maskowanie

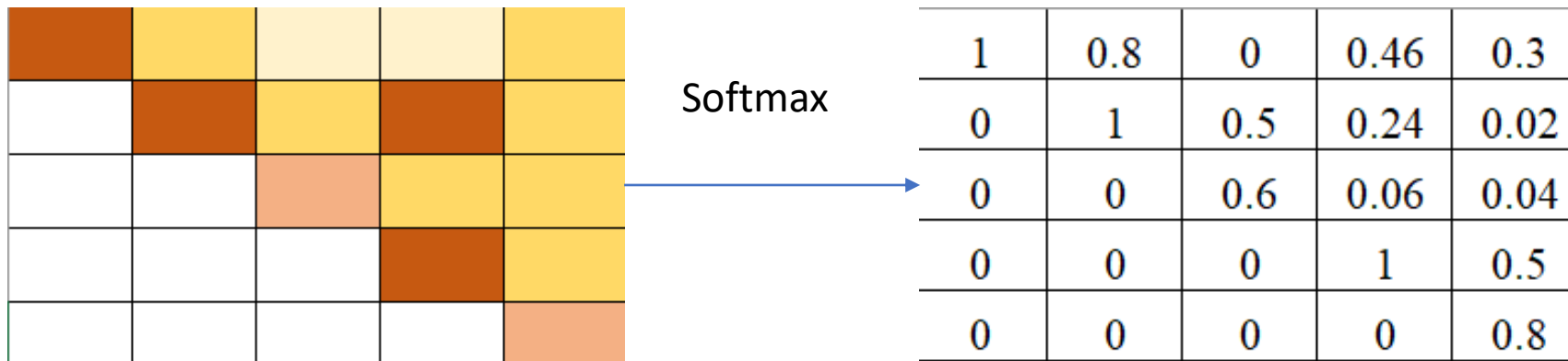
$$Attention(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

				Token1	Token2	Token3	Token4	Token5
				E1	E2	E3	E4	E5
				E1WQ	E2 WQ	E3 WQ	E4 WQ	E5 WQ
				Q1	Q2	Q3	Q4	Q5
Token1	E1	E1 WK	K1	K1Q1	K1Q2	K1Q3	K1Q4	K1Q5
Token2	E2	E2 WK	K2	K2Q1	K2Q2	K2Q3	K2Q4	K2Q5
Token3	E3	E3 WK	K3	K3Q1	K3Q2	K3Q3	K3Q4	K3Q5
Token4	E4	E4 WK	K4	K4Q1	K4Q2	K4Q3	K4Q4	K4Q5
Token5	E5	E5 WK	K5	K5Q1	K5Q2	K5Q3	K5Q4	K5Q5
Token6	E6	E6 WK	K6	K6Q1	K6Q2	K6Q3	K6Q4	K6Q5
Token7	E7	E7 WK	K7	K7Q1	K7Q2	K7Q3	K7Q4	K7Q5
Token8	E8	E8 WK	K8	K8Q1	K8Q2	K8Q3	K8Q4	K8Q5
Token9	E9	E9 WK	K9	K9O1	K9O2	K9O3	K9O4	K9O5

4. Maskowanie

$$\text{Attention}(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Podczas obliczania wyniku samouwagi maskowanie jest stosowane do licznika tuż przed Softmax.
- Zamaskowane elementy (białe kwadraty) są ustawione na ujemną nieskończoność, więc Softmax zamienia te wartości na zero.



Znormalizowana macierz samouwagi

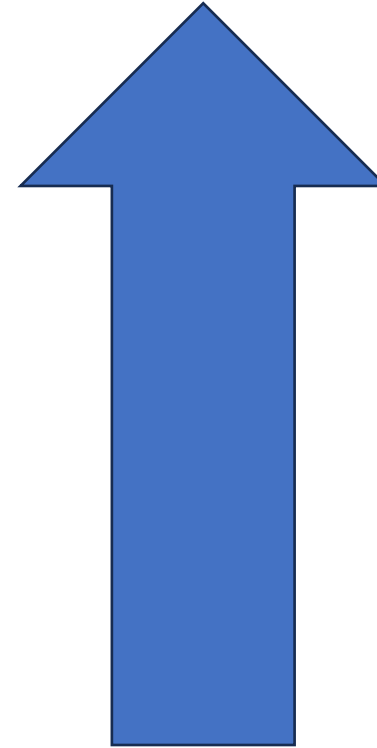
5. Obliczenie połączeń resztowych

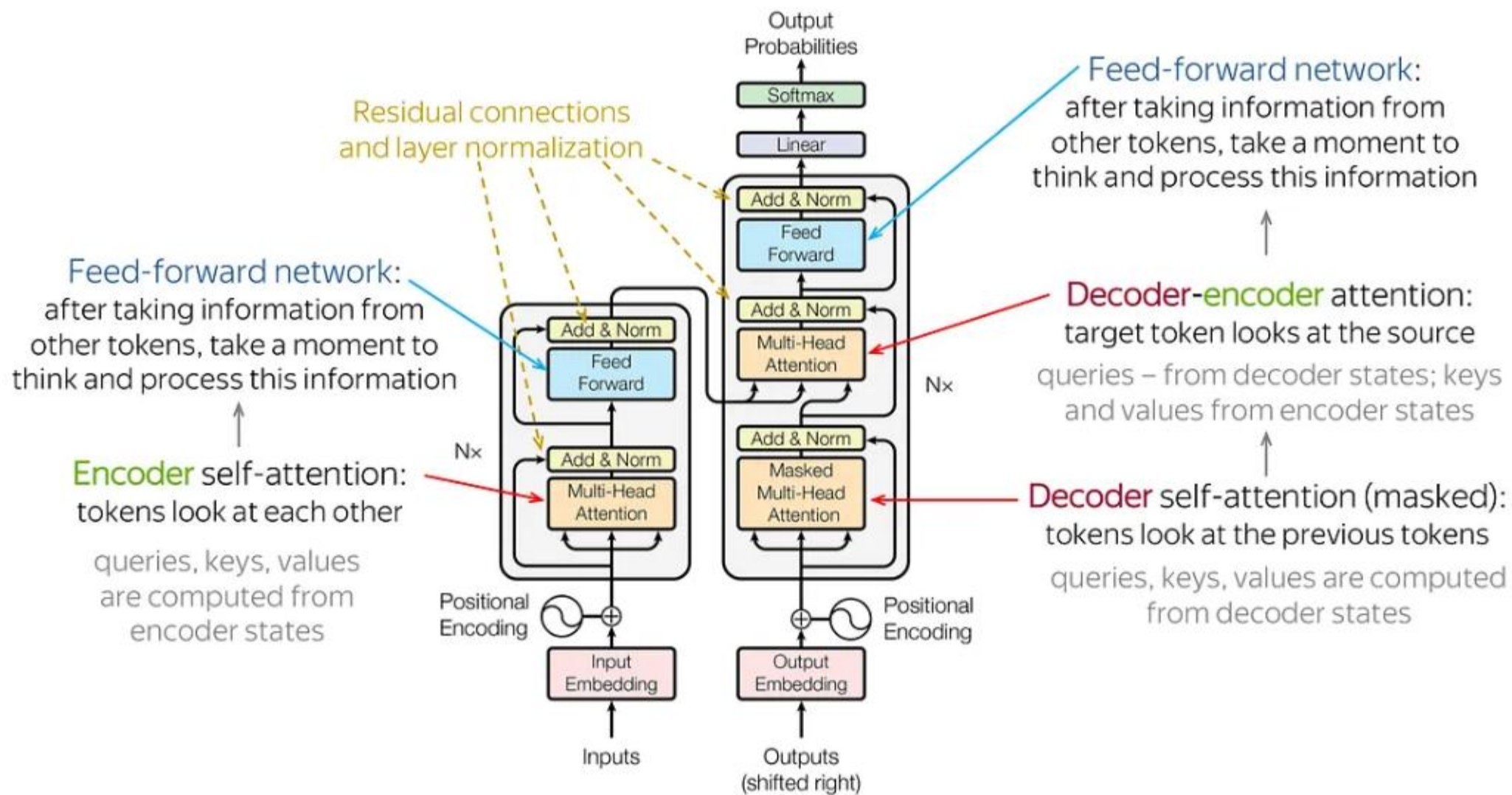
6. Przejęcie przez FFN

7. Wykorzystanie funkcji Softmax do predykcji następnego tokenu.

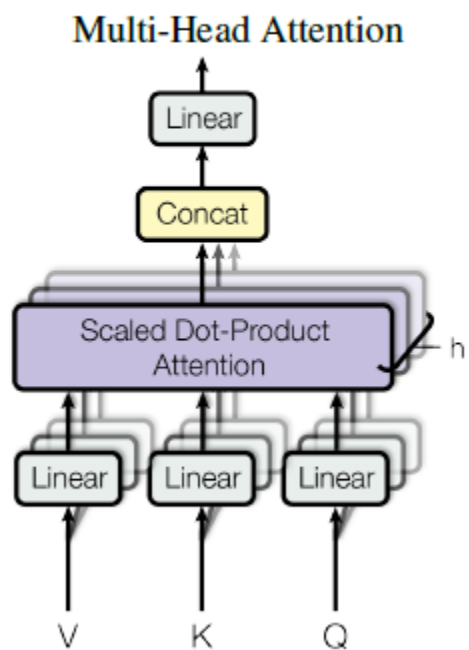
Schemat

- Sieć gęsto połączona
- Połączenia resztowe
- Obliczanie samo-uwagi (self-Attention)
- Osadzenie słów i pozycji
- Osadzanie słów (Word embedding)





[Image Source: [Cross Validated](#)]



```
CLASS torch.nn.Transformer(d_model=512, nhead=8, num_encoder_layers=6,  
num_decoder_layers=6, dim_feedforward=2048, dropout=0.1,  
activation=<function relu>, custom_encoder=None,  
custom_decoder=None, layer_norm_eps=1e-05, batch_first=False,  
norm_first=False, bias=True, device=None, dtype=None) \[SOURCE\]
```

`nn.MultiheadAttention`

`scaled_dot_product_attention`

torch.nn.attention

Elementy historii



GPT- Generative Pre-trained Transformer

Pierwotny model GPT OpenAI („GPT-1”) 2018 r

GPT-2 - 2019 nienadzorowany model języka

Model językowy GPT-3, 2020 r.

Model językowy GPT-3.5

Uruchomienie ChatGPT3- listopad/grudzień 2022 r. –
popularyzacja Transformersów i LLM

DALL-E, 2021, model głębokiego uczenia, który może
generować cyfrowe obrazy z opisów w języku naturalnym

BERT

Bidirectional Encoder Representations from Transformers (BERT) to model językowy oparty na architekturze transformersów, wyróżniający się znaczną poprawą w stosunku do poprzednich modeli.

Został on wprowadzony w październiku 2018 r. przez naukowców z Google.

2015–2018 OpenAI

W grudniu 2015, w San Francisco, ogłoszono założenie OpenAI non-profit. Grupę inwestorów – założycieli stanowili: Sam Altman, Greg Brockman, Reid Hoffman, Jessica Livingston, Peter Thiel, Elon Musk, Amazon Web Services (AWS), Infosys i YC Research, którzy zadeklarowali wpłatę łącznej kwoty 1 miliarda dolarów amerykańskich na założenie organizacji.

Do inwestorów dołączyła grupa dziewięciu światowej klasy naukowców – założycieli: Ilya Sutskever, Greg Brockman, Trevor Blackwell, Vicki Cheung, Andrej Karpathy, Durk Kingma, John Schulman, Pamela Vagata i **Wojciech Zaremba**.

W grupie doradców znaleźli się: Pieter Abbeel, Yoshua Bengio, Alan Kay, Sergey Levine i Vishal Sikka. Na czele grupy kreatorów przedsięwzięcia stanęli Sam Altman i Elon Musk.



Wikipedia

<https://en.wikipedia.org> › wiki · [Tłumaczenie strony](#) ⋮

Wojciech Zaremba

Wojciech Zaremba (born 30 November 1988) is a Polish computer scientist and founding team member of OpenAI (2016–present).

Studiował na [Uniwersytecie Warszawskim](#) i [École polytechnique](#)

Elementy historii

GPT-3: Its Nature, Scope, Limits, and Consequences

Luciano Floridi^{1,2} · Massimo Chiriatti³

Published online: 1 November 2020

© The Author(s) 2020

Wywiad z Wojciechem Zarembą:

<https://www.youtube.com/watch?v=6QhGUQ5iTdk&t=5s>

<https://newsletter.artofsaience.com/p/vision-transformers-from-idea-to>
<https://jalammar.github.io/illustrated-transformer/>

Architektura GPT3

Model oparty na dekodерze

Zastosowane maskowanie

175 miliardów parametrów dokładnie 175 181 291 520

27 938 macierzy

Wektory osadzenia: wymiar 12 288

(każdy wektor osadzenia (embedding) ma 12 288 współrzędnych)

Wektory Key i Query mają 128 współrzędnych

Kontekst: 2048 tokenów (context size)-jednorazowo przez sieć przechodzi 2048 tokenów.

Ostatnia warstwa-output 50 257 słów (wymiar słownika)

<https://www.youtube.com/watch?v=wjZofJX0v4M> 3Blue1Brown

Transformers (how LLMs work) explained visually | DL5



3Blue1Brown ✓
7,23 mln subskrybentów



Subskrybujesz ▾

Macierze W_Q , W_V i W_K mają 128 wierszy i 12 288 kolumn

GPT-3		Total weights:
		175,181,291,520
Embedding	$\overset{12,288}{d_embed} * \overset{50,257}{n_vocab}$	= 617,558,016
Key	$\overset{128}{d_query} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Query	$\overset{128}{d_query} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Value	$\overset{128}{d_value} * \overset{12,288}{d_embed} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Output	$\overset{12,288}{d_embed} * \overset{128}{d_value} * \overset{96}{n_heads} * \overset{96}{n_layers}$	= 14,495,514,624
Up-projection	$\overset{49,152}{n_neurons} * \overset{12,288}{d_embed} * \overset{96}{n_layers}$	= 57,982,058,496
Down-projection	$\overset{12,288}{d_embed} * \overset{49,152}{n_neurons} * \overset{96}{n_layers}$	= 57,982,058,496
Unembedding	$\overset{50,257}{n_vocab} * \overset{12,288}{d_embed}$	= 617,558,016

Transformer models: an introduction and catalog

Xavier Amatriain xavier@amatriain.net

Ananth Sankar ansankar@linkedin.com

Jie Bing jbing@linkedin.com

Praveen Kumar Bodigutla pbodigutla@linkedin.com

Timothy J. Hazen thazen@linkedin.com

Michael Kazi mkazi@linkedin.com

April 2, 2024

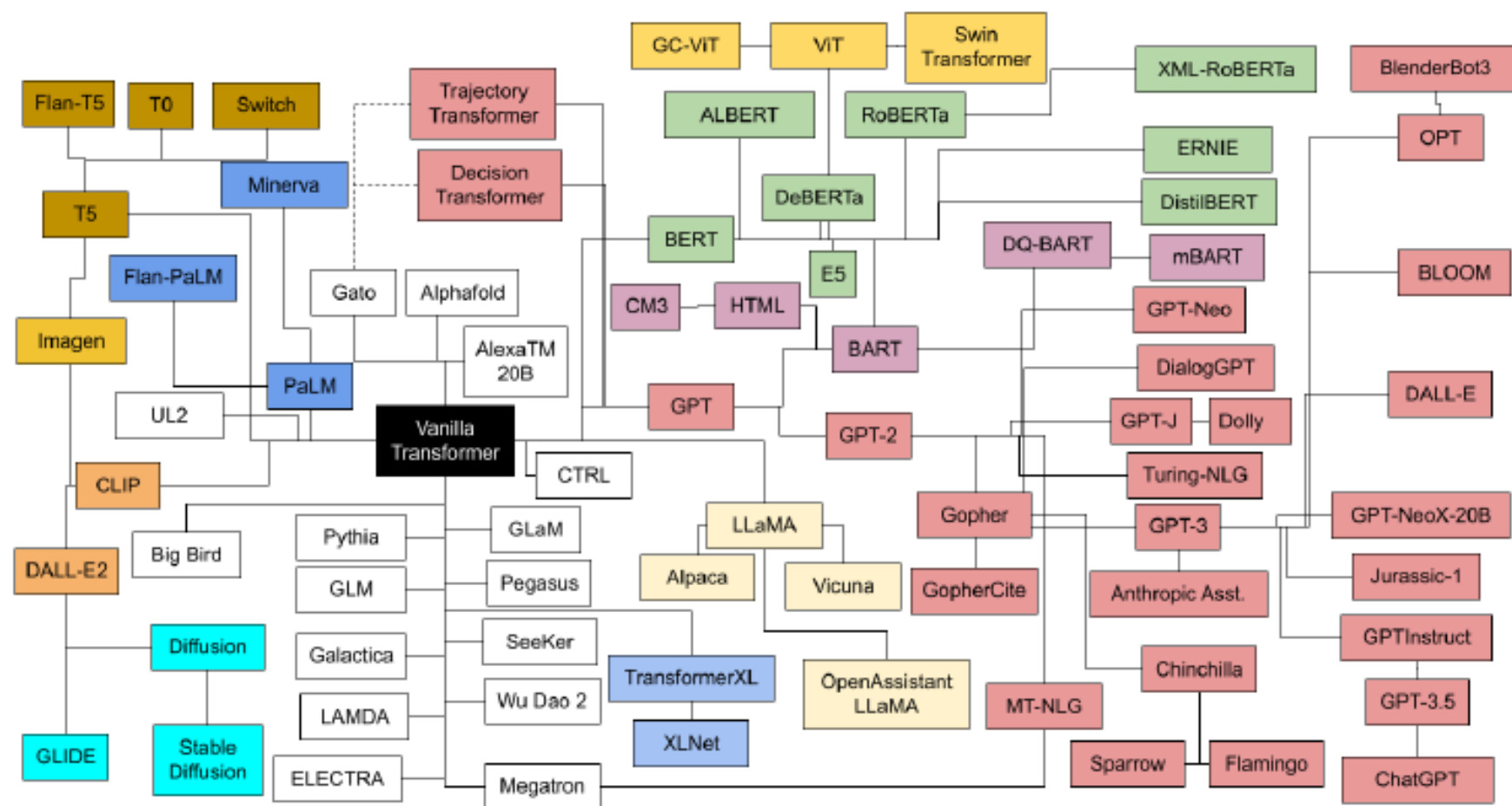


Figure 5: Transformers Family Tree

Źródło: Transformer models: an introduction and catalog

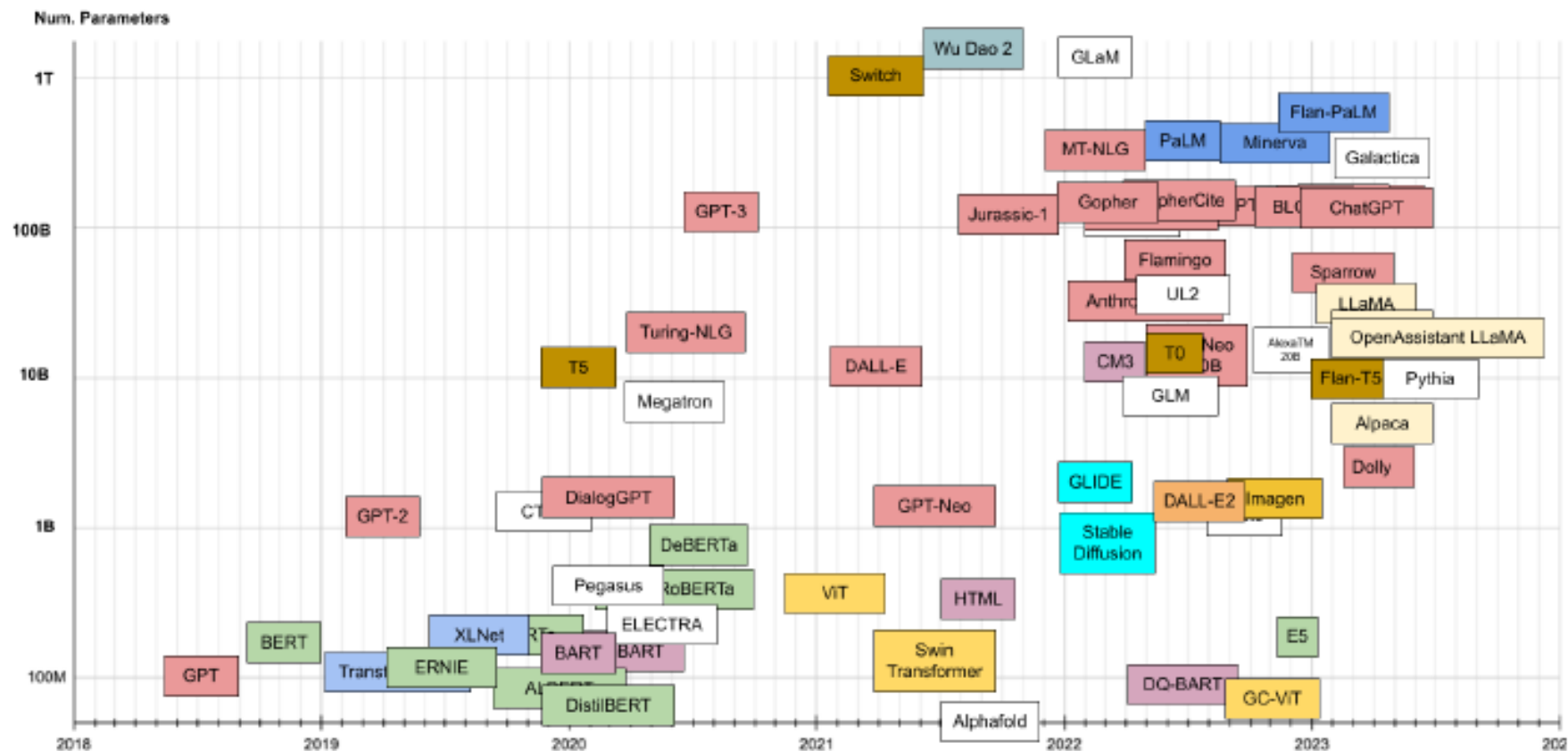


Figure 7: Transformer timeline. On the vertical axis, number of parameters. Colors describe Transformer family.

Źródło: Transformer models: an introduction and catalog

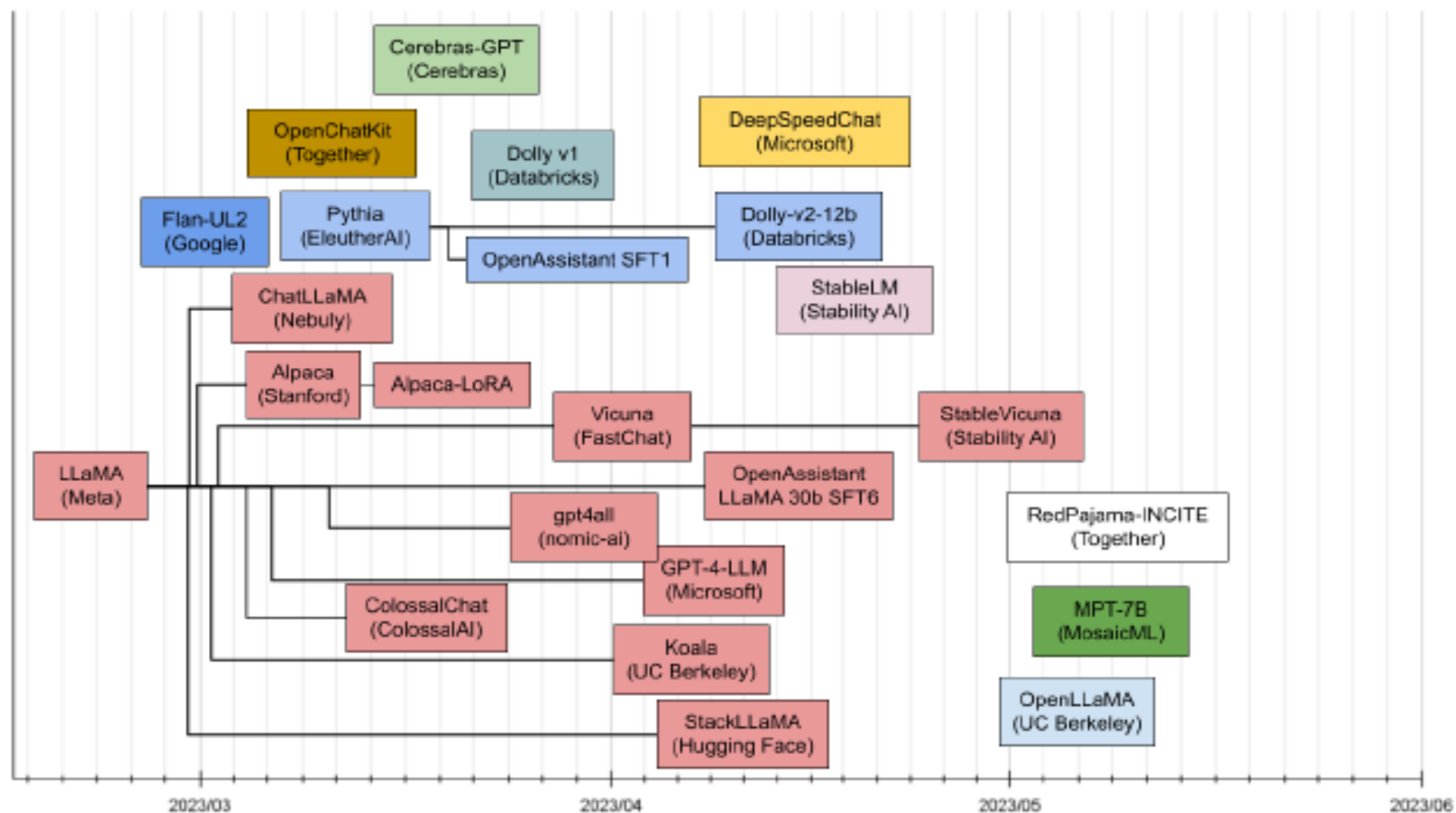
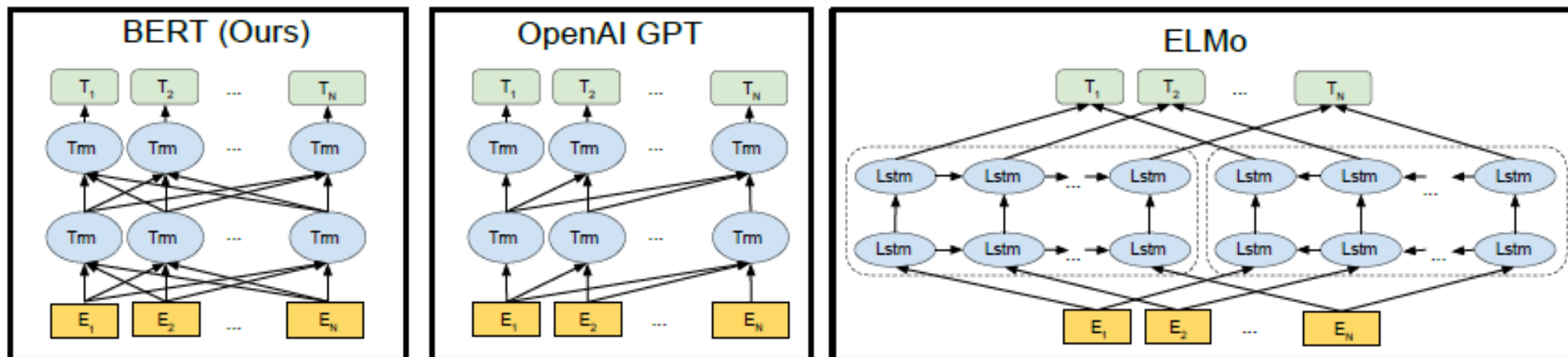


Figure 8: Recently published LLMs

Źródło: Transformer models: an introduction and catalog



Różnice w architekturze modeli pretrained.

- BERT wykorzystuje dwukierunkowy transformer.
- OpenAI GPT wykorzystuje transformer od lewej do prawej.
- ELMo wykorzystuje połączenie niezależnie wytrenowanych modeli LSTM od lewej do prawej i od prawej do lewej do generowania funkcji dla dalszych zadań.

Spośród tych trzech, tylko reprezentacje BERT są wspólnie warunkowane zarówno lewym, jak i prawym kontekstem we wszystkich warstwach.

Źródło: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Miara dobroci dopasowania modeli NLP

Na przestrzeni lat opracowano wiele różnych wskaźników NLP.

Jednym z najbardziej popularnych jest Bleu Score.

Wyniki Bleu mieszczą się w przedziale od 0 do 1.

Wynik 0,6 lub 0,7 jest uważany za najlepszy z możliwych.

<https://towardsdatascience.com/foundations-of-nlp-explained-bleu-score-and-wer-metrics-1a5ba06d812b>

Miara BLEU

BLEU Score został zaproponowany przez Kishore Papineni i in. w ich artykule z 2002 r. pt. Method for Automatic Evaluation of Machine Translation.

Podejście to polega na zliczaniu dopasowanych n-gramów w tłumaczeniu do n-gramów w tekście referencyjnym, gdzie 1-gram lub unigram to każdy token, a porównanie bigramów to każda para słów. Porównanie jest dokonywane niezależnie od kolejności słów.

Biblioteka Python **Natural Language Toolkit** (NLTK) zapewnia implementację miary BLEU, której można użyć do oceny wygenerowanego tekstu względem odniesienia.

Wskaźnik BLEU dla zdania:

```
from nltk.translate.bleu_score import sentence_bleu
reference = [['this', 'is', 'a', 'test'], ['this', 'is', 'test']]
candidate = ['this', 'is', 'a', 'test']
score = sentence_bleu(reference, candidate)
print(score)
```

```
[6]: ## https://github.com/sdadas/polish-nlp-resources?tab=readme-ov-file#word2vec
##!pip install transformers
from transformers import pipeline
```

```
generator = pipeline("text-generation", model="sdadas/polish-gpt2-medium")
results = generator(seed_text1,
                    max_new_tokens=50, do_sample=True, repetition_penalty = 1.2,
                    num_return_sequences=1, num_beams=1, temperature=0.95, top_k=50, top_p=0.95
                    )
```

Setting `pad\_token\_id` to `eos\_token\_id`:2 for open-end generation.



Sławomir Dadas

National Information Processing Institute, Warsaw, Poland
Zweryfikowany adres z opi.org.pl

machine learning natural language processing

Text oryginalny:

na stronę liberalizmu były dwa odcienie liberalizmu umiarkowany i radykalny chcący przeprowadzić zasadniczą reformę konstytucji do tego radykalnego kierunku przyłączył się bentham w r sam zredagował plan reformy przystąpienie filozofów związanie utylitaryzmu z radykalizmem miało swe uzasadnienie w naczelnej idei utylitaryzmu zgodności interesów w społeczeństwie zgodność interesów może być naturalna jak

Text starter:

na stronę liberalizmu były dwa odcienie liberalizmu umiarkowany i radykalny chcący przeprowadzić zasadniczą reformę konstytucji do tego radykalnego kierunku przyłączył się bentham w r sam zredagował pla

<https://huggingface.co/sdadas>

Text wygenerowany na podstawie startera:

r sam zredagował placet (i to wbrew woli większości swoich wyborców) a na koniec dodał taki jeden z niewielu projektów. I co zrobili te 4 partie? Wszystkie jak pojebane wyklóciły się z nim o jego program gospodarczy, tylko że w żaden sposób nie chcieli go

Modele języka w Polsce

<https://clip.ipipan.waw.pl/>

PLLuM



PLLuM

Polish Large Language Model

Logo PLLuM

Autor	Ministerstwo Cyfryzacji
Pierwsze wydanie	24 lutego 2025
Rodzaj	duży model językowy
Licencja	CC-BY-NC-4.0, Apache License 2.0, Llama 3.3

Mistral AI



Porozmawiaj z Bielikiem



Bielik.AI – europejska rodzina otwartych modeli językowych, stworzona w Polsce.

Otwarte, bezpłatne i bezpieczne. Rozmawiaj z Bielikiem, eksperymentuj i buduj własne rozwiązania

Korzystaj z Bielika

Zainstaluj Bielika lokalnie



Polski Bielik obnaża ograniczenia ChatGPT || Remigiusz Kinas - didaskalia#170



didaskalia

419 tys. subskrybentów



4,8 tys.



Udostępnij



Zapisz

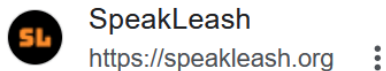
192 tys. wyświetleń 2 miesiące temu Didaskalia

Ponowne odtwarzanie czatu na żywo



Bielik to polski otwarty model językowy
stworzony przez zespół SpeakLeash we
współpracy z ACK Cyfronet AGH.

Krzysztof Ociepa – współzałożyciel Azurro
– nadzoruje w zespole SpeakLeash trening i
fine-tuning Bielika i wie o nim dosłownie
wszystko. Dzięki dogłębnej wiedzy na temat
modelu jesteśmy w stanie zaproponować
wykorzystanie Bielika dopasowane do
potrzeb klienta.



SpeakLeash | Spichlerz

Naszym celem jest umożliwienie badań nad uczeniem maszynowym i wytrenowanie na podstawie zebranych danych, generatywnego, wstępnie wytrenowanego modelu ...

Menu
Tools and resources
Research centers
Projects
Wiki
Find page
Recent changes
Help

National Corpus of Polish

The National Corpus of Polish is a shared initiative of four institutions: Institute of Computer Science at the Polish Academy of Sciences (coordinator), Institute of Polish Language at the Polish Academy of Sciences, Polish Scientific Publishers PWN, and the Department of Computational and Corpus Linguistics at the University of Łódź. It has been carried out as a research-development project of the Ministry of Science and Higher Education.

These four institutions have started cooperation to build a reference corpus of Polish language containing over fifteen hundred millions of words. The list of sources for the corpora contains classic literature, daily newspapers, specialist periodicals and journals, transcripts of conversations, and a variety of short-lived and internet texts. For a corpus to be reliable, not only it is necessary to contain a high number of words, but it also needs a diversity of texts with respect to the subject and genre. The conversations ought to represent both male and female speakers, in various age groups, coming from various regions in Poland.

[Official project website](#)

Manually annotated subcorpus

The manually annotated 1-million word subcorpus of the National Corpus of Polish, available on CC-BY licence (NKJP tagset):

- [NKJP-PodkorpusMilionowy-1.2.tar.gz](#) — GZip compressed tar archive,
- [NKJP-PodkorpusMilionowy-1.2.tar.bz2](#) — BZip2 compressed tar archive.

XCES encoded version

The manually annotated subcorpus, encoded using the XCES format (NKJP tagset).

- [nkjp1m-1.2-xces-xml.bz2](#) — XCES encoded corpus; [here](#) you may find the order of the source directories in the combined file,
- [nkjp1m-1.2-xces-reanalysed-sgjp-xml.bz2](#) — XCES encoded corpus, reanalysed according to the procedure described [here](#), using Morfeusz SGJP,
- [nkjp1m-1.2-xces-reanalysed-polimorf-xml.bz2](#) — XCES encoded corpus, reanalysed according to the procedure described [here](#), using Morfeusz Polimorf.

SGJP tagset version

The manually annotated subcorpus, converted to the newest version of the SGJP tagset.

- Newest version is automatically created after any changes to the SGJP tagset are made. This version is released at the [SGJP website](#).

Tagger models, trained on the latest version of the subcorpus

- Model for Concraft 2.0 is available at its [official GitHub page](#).
- [concraft-model-nkjp1m-1.2.gz](#) — model for Concraft 0.7.1 tagger,
- [pantera-model-nkjp1m-1.2.bz2](#) — model for PANTERA 0.9.1 tagger,
- [wcrft-model-nkjp-1.2.tar.gz](#) — model for WCRFT1 and WCRFT2 taggers,
- [wmbt-model-nkjp1m-1.2.tar.bz2](#) — model for WMBT tagger.

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

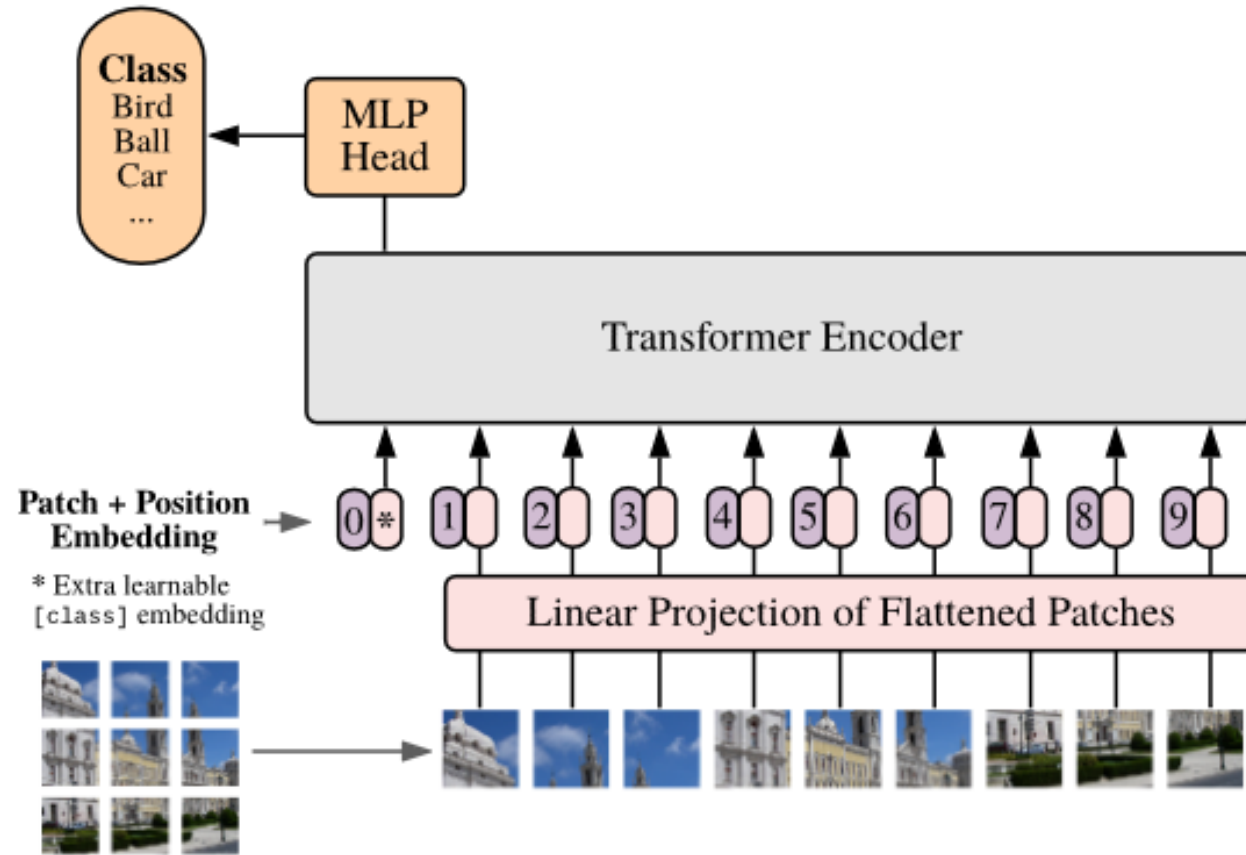
**Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}**

^{*}equal technical contribution, [†]equal advising

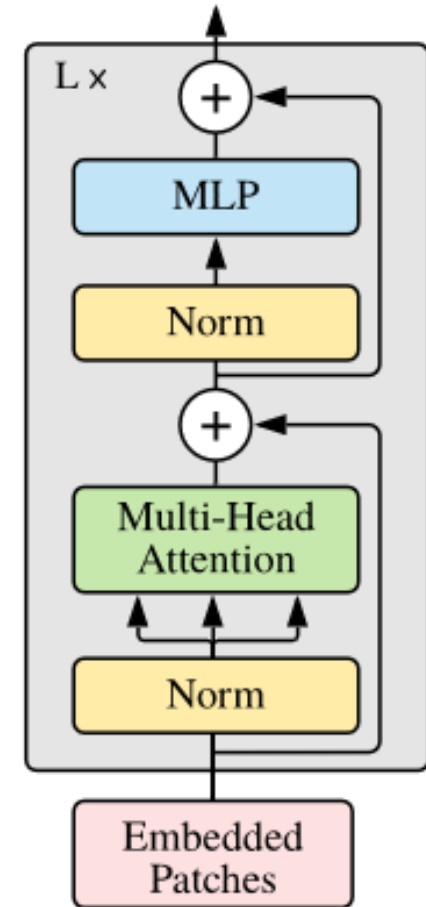
Google Research, Brain Team

`{adosovitskiy, neilhoulby}@google.com`

Vision Transformer (ViT)



Transformer Encoder



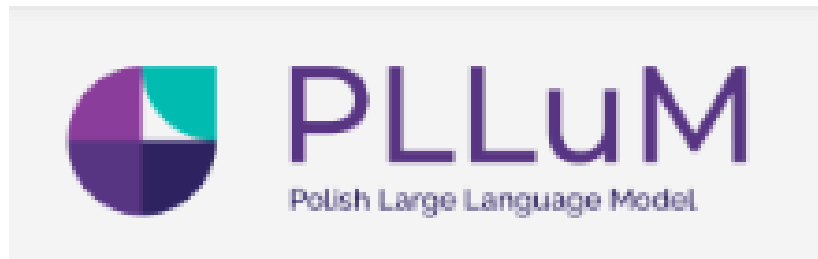
How DeepSeek Rewrote the Transformer [MLA]

- https://www.youtube.com/watch?v=0VLAoVGf_74&t=44s

LLM-duże modele językowe

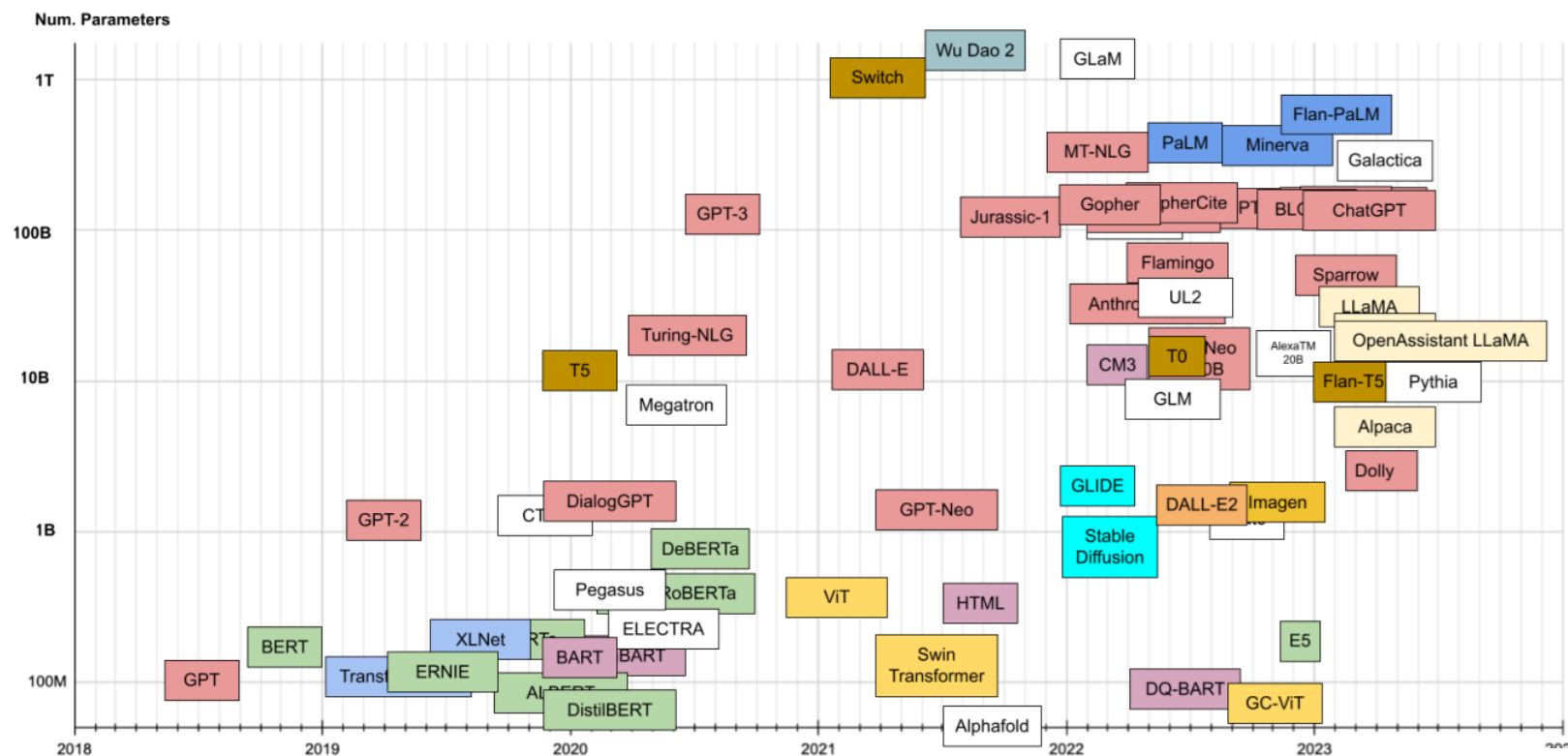
<https://zapier.com/blog/best-llm/>

<https://www.shakudo.io/blog/top-9-large-language-models>



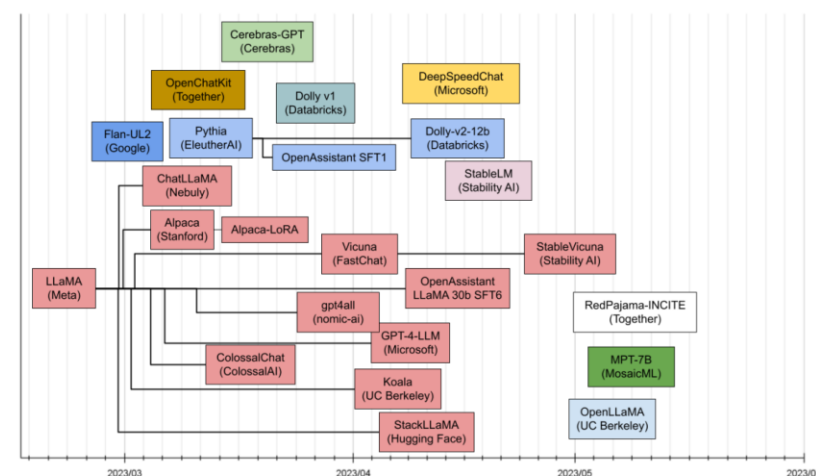
1. GPT
2. DeepSeek
3. Qwen
4. Grok
5. LLaMA
6. Claude
7. Mistral
8. Gemini
9. Qwen

LLM	Developer
GPT-4o	OpenAI
o3 and o1	OpenAI
Gemini	Google
Gemma	Google
Llama	Meta
R1	DeepSeek
V3	DeepSeek
Claude	Anthropic
Command	Cohere
Nova	Amazon
Large 2	Mistral AI
Qwen	Alibaba Cloud



Transformer timeline. On the vertical axis, number of parameters

<https://llm-timeline.com/>



Recently published LLMs

<https://sebastianraschka.com/llm-architecture-gallery/>

Sebastian **Raschka**      

 [Blog](#) [Books](#) [Courses](#) [LLM Gallery](#) [LLMs From Scratch](#) [Reasoning Models](#) [Talks](#)

LLM Architecture Gallery

Last updated: April 10, 2026 [\(view changes\)](#)  [RSS](#)

If you do not see the latest changes, try a hard reload: `Cmd+Shift+R` on Mac or `Ctrl+F5` on Windows.

This page collects architecture figures and fact sheets from posts on [my blog](#), plus selected release posts or technical reports when a new architecture has not been covered there yet. Click a figure to enlarge it, or use the model title to jump to the source article.

If you spot an inaccurate fact sheet, mislabeled architecture, or broken link, please file an issue here: [Architecture Gallery issue tracker](#).

<https://sebastianraschka.com/llm-architecture-gallery/>

MODEL A

Qwen3.5 (397B)

MODEL B

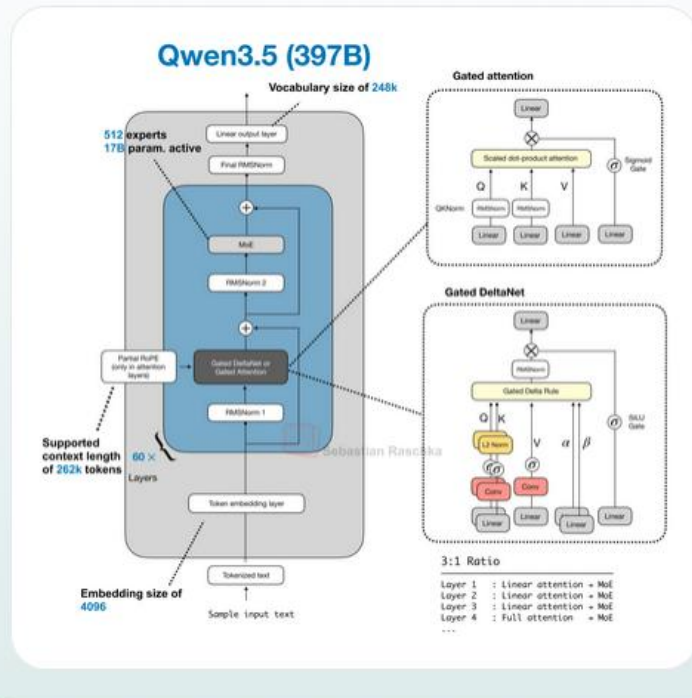
DeepSeek V3.2 (671B)

Swap

Clear

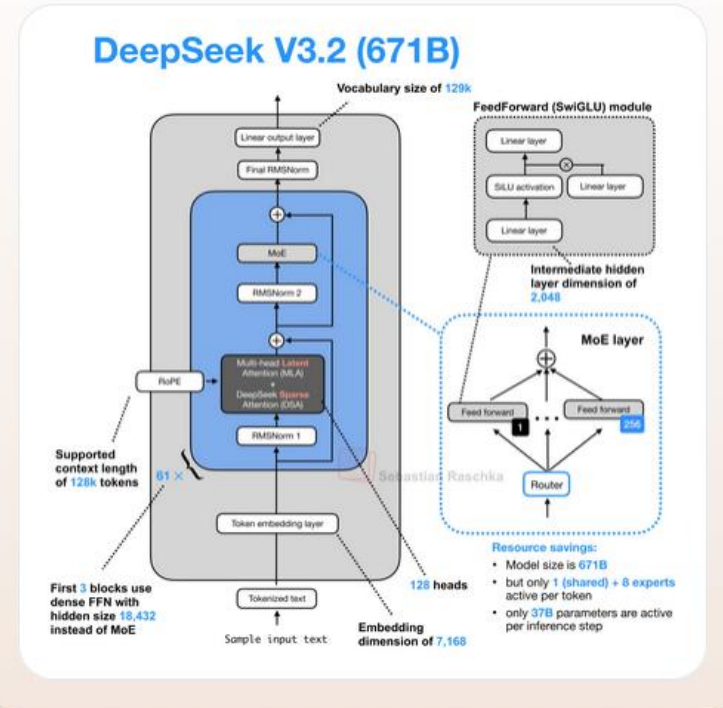
MODEL A

Qwen3.5 (397B)

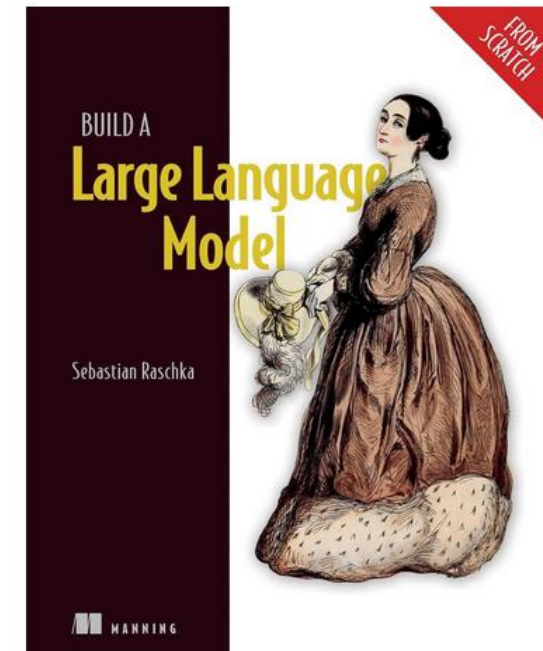
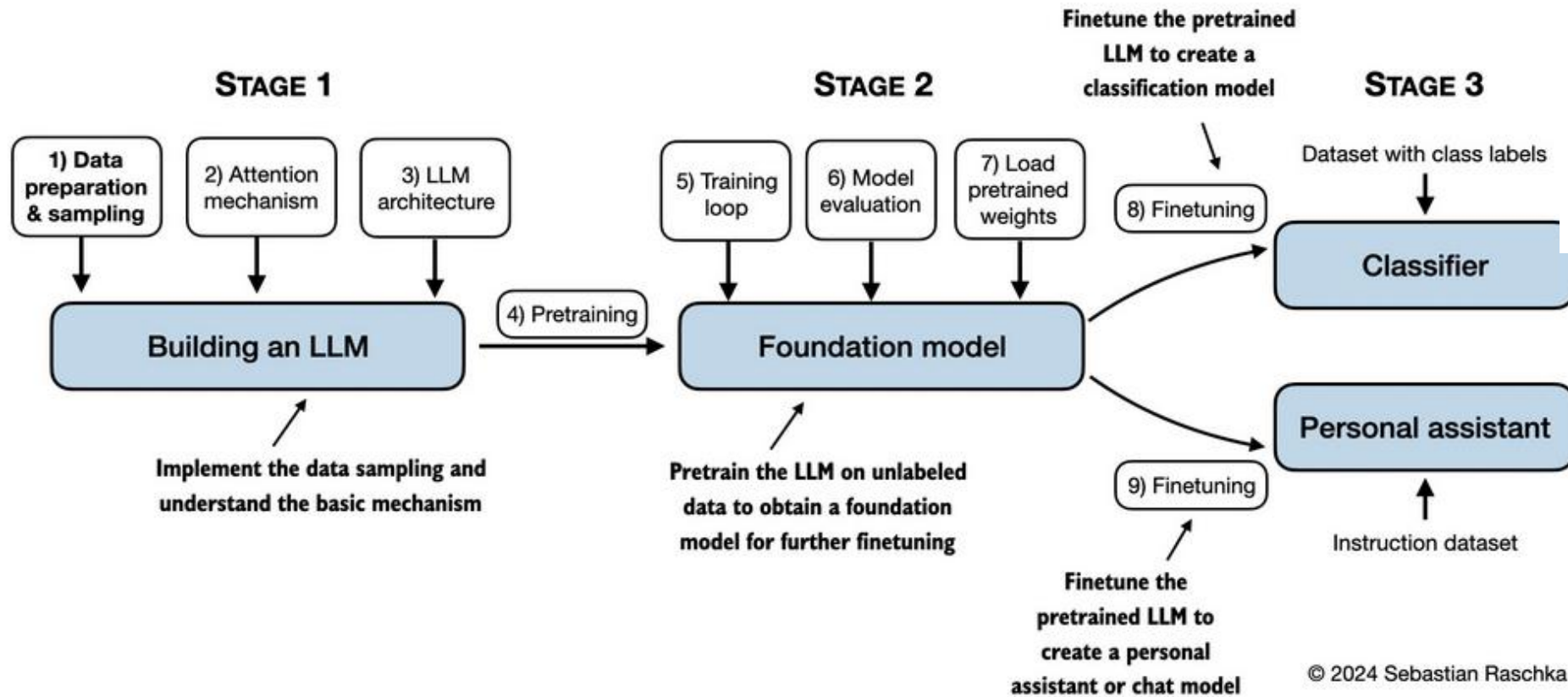


MODEL B

DeepSeek V3.2 (671B)



Sebastian Raschka





MIT 6.S191: Deep Generative Modeling

Alexander Amini • 38 tys. wyświetleń • 4 tygodnie temu



MIT 6.S191: Reinforcement Learning

Alexander Amini • 16 tys. wyświetleń • 3 tygodnie temu



MIT 6.S191: Language Models and New Frontiers

Alexander Amini • 20 tys. wyświetleń • 2 tygodnie temu



MIT 6.S191 (Google): Large Language Models

Alexander Amini • 27 tys. wyświetleń • 8 dni temu



MIT 6.S191 (Liquid AI): Large Language Models

Alexander Amini • 8,8 tys. wyświetleń • 1 dzień temu

To jest w programie-zrobimy w maju

Peter Grabowski leads an applied research team focused on Gemini at Google.

28-30 minuta

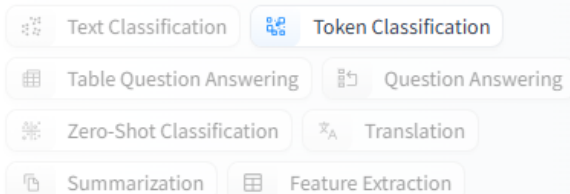
Instruction Tuning

Reinforcement
Learning with
Human Feedback

35 minuta AI agents

LLM-baza do wielu zastosowań

Natural Language Processing



Davlan/bert-base-multilingual-cased-ner-hrl

Token Classification • Updated Nov 11, 2024 • ↓ 404k • ♥ 72

Jean-Baptiste/camembert-ner-with-dates

Token Classification • Updated Jun 16, 2023 • ↓ 208k • ♥ 43

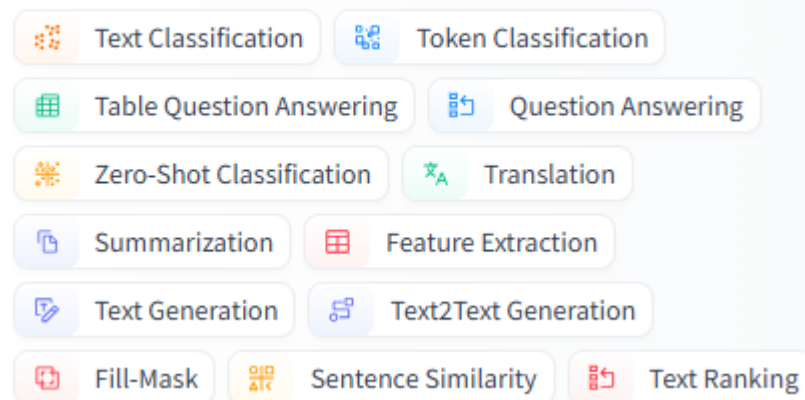
HooshvareLab/bert-fa-base-uncased-ner-peyma

Token Classification • Updated May 18, 2021 • ↓ 46.2k • ♥ 7

Jean-Baptiste/camembert-ner

Token Classification • Updated Jun 1, 2023 • ↓ 242k • ♥ 110

Natural Language Processing



Zastosowanie LLM

- Czatboty ogólnego przeznaczenia (takie jak ChatGPT i Google Gemini)
- Podsumowywanie wyników wyszukiwania i innych informacji z sieci
- Czatboty obsługi klienta, które są szkolone na dokumentach i danych firmy
- Tłumaczenie tekstu z jednego języka na inny
- Konwersja tekstu na kod komputerowy lub tekstu z jednego języka na inny
- Generowanie postów w mediach społecznościowych, postów na blogu i innych tekstów marketingowych
- Analiza sentymentów
- Moderowanie treści
- Poprawianie i edytowanie tekstów
- Analiza danych

- <https://pixelplex.io/blog/llm-applications/>

Top 10 applications of large language models



1. **Content generation**
2. **Translation and localization**
3. **Search and recommendation**
4. **Virtual assistants**
5. **Code development**
6. **Sentiment analysis**
7. **Question answering**
8. **Market research**
9. **Education**
10. **Classification**

LLM-jak działają?

LLM-dane-korpusy

O korpusie

Korpus to dowolny zbiór tekstów, w którym czegoś szukamy...

Korpus tekstów musi być odpowiednio zrównoważony gatunkowo, chronologicznie, stylowo, terytorialnie i pod innymi względami, np. ze względu na wiek i płeć autorów. Rodzaj zrównoważenia korpusu zależy od celów, jakim korpus służy.

Nasz korpus tekstów polskich to fragment słownikowej kuchni, czyli autentyczny materiał językowy, na którego podstawie opisujemy znaczenia słów i konstrukcji. Pojedyncze zdania z tekstów korpusu zamieszczamy w słownikach jako przykłady ilustrujące znaczenia.

Korpus wykorzystywany do tworzenia słowników ogólnych powinien gromadzić teksty z różnych dziedzin tematycznych, stylów i źródeł.

Zrównoważony tematycznie i gatunkowo **Korpus Języka Polskiego PWN** liczy **70 milionów słów**. Cały korpus, włączając archiwa prasowe i klasykę literacką od średniowiecza, **zawiera 100 milionów słów**. Składa się z tekstów książek, czasopism, druków ulotnych i akcydensowych (np. reklam, instrukcji obsługi, regulaminów, ulotek wyborczych), stron internetowych oraz tekstów mówionych. W porównaniu z innymi korpusami na świecie nasz zbiór zawiera dość dużo tekstów literackich. Postanowiliśmy bowiem uwzględnić szczególnie żywą w Polsce tradycję autorytetu kulturalnego jako kryterium poprawności językowej.

WebText

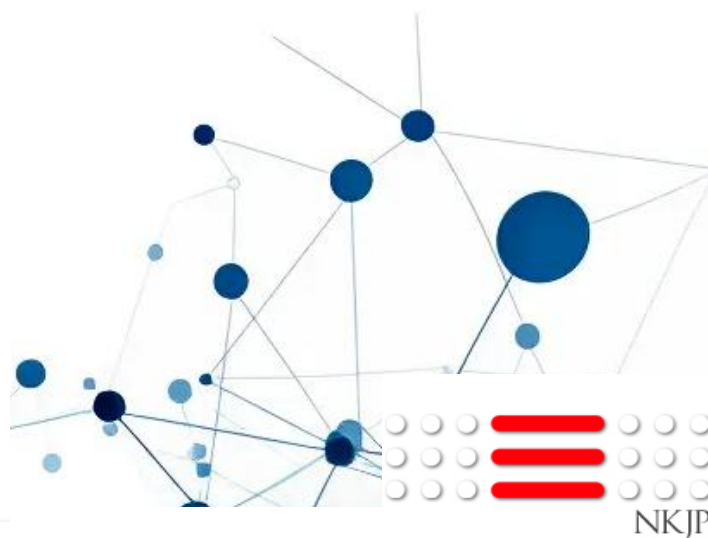
Introduced by Radford et al. in [Language Models are Unsupervised Multitask Learners](#)

WebText is an internal OpenAI corpus created by scraping web pages with emphasis on document quality. The authors scraped all outbound links from Reddit which received at least 3 karma. The authors used the approach as a heuristic indicator for whether other users found the link interesting, educational, or just funny.



OpenWebText

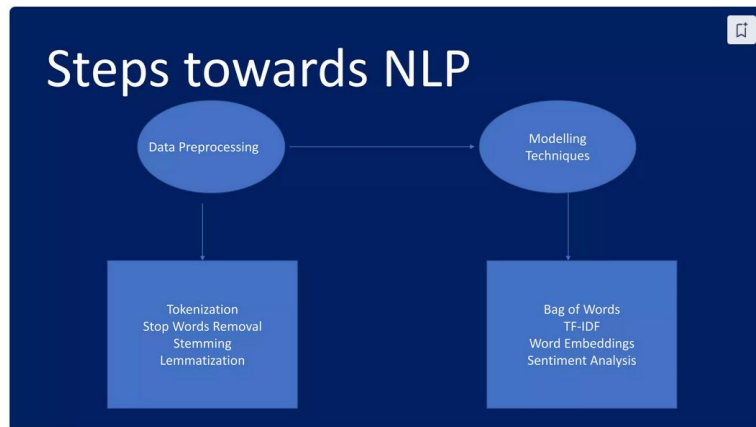
Introduced by Aaron Gokaslan et al. in [OpenWebText corpus](#)



Hugging Face

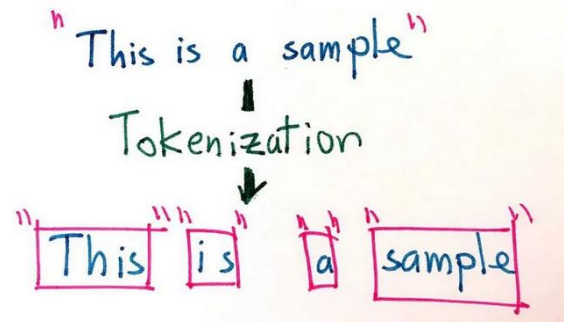
NATIONAL CORPUS OF
POLISH

LLM-jak działają?



<https://www.slideshare.net/slideshow/nlp-pptptx/252713630>

Czyszczenie tekstu

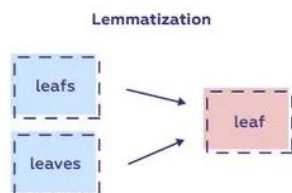
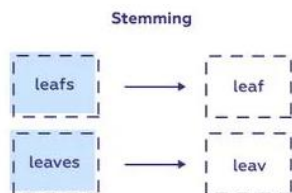


Resource: <https://medium.com/data-science-in-your-pocket/tokenization-algorithms-in-natural-language-processing-nlp-ffceab8454af>

Stop Words Example

Sample Text with Stop Words	Sample Text without Stop Words
Aarush Coaching Classes – A stem learning place for kids	Aarush Coaching Classes, Stem, Learning, Place, kids
Can Listening be exhausting ?	Listening, Exhausting
I like Teaching, so I teach	Like, Teaching, Teach

<https://www.slideshare.net/slideshow/nlp-pptptx/252713630>



sciforce

Resource: <https://tr.pinterest.com/pin/706854104005417976/>



Resource: <https://blog.aaronccwong.com/2019/building-a-bigram-hidden-markov-model-for-part-of-speech-tagging/>

Packages supporting Polish language text clearing

Sentence analysis (tokenization, lemmatization, POS tagging etc.)

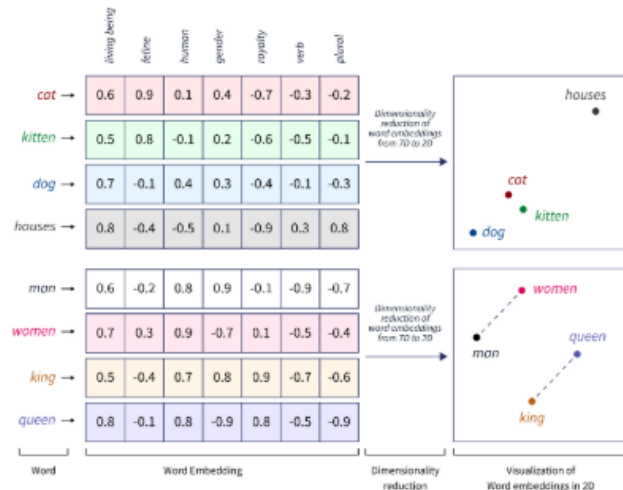
- SpaCy - A popular library for NLP in Python which includes Polish models for sentence analysis.
- Stanza - A collection of neural NLP models for many languages from Stanford NLP.
- Trankit - A light-weight transformer-based python toolkit for multilingual natural language processing by the University of Oregon.
- KRNNT and KFTT - Neural morphosyntactic taggers for Polish.
- Morfeusz - A classic Polish morphosyntactic tagger.
- Language Tool - Java-based open source proofreading software for many languages with sentence analysis tools included.
- Stempel - Algorithmic stemmer for Polish.
- PoLemma - pIT5-based lemmatizer of named entities and multi-word expressions for Polish, available in small, base and large sizes.

LLM-jak działają?

Uczenie modelu na dużych zbiorach danych (korpora językowe).

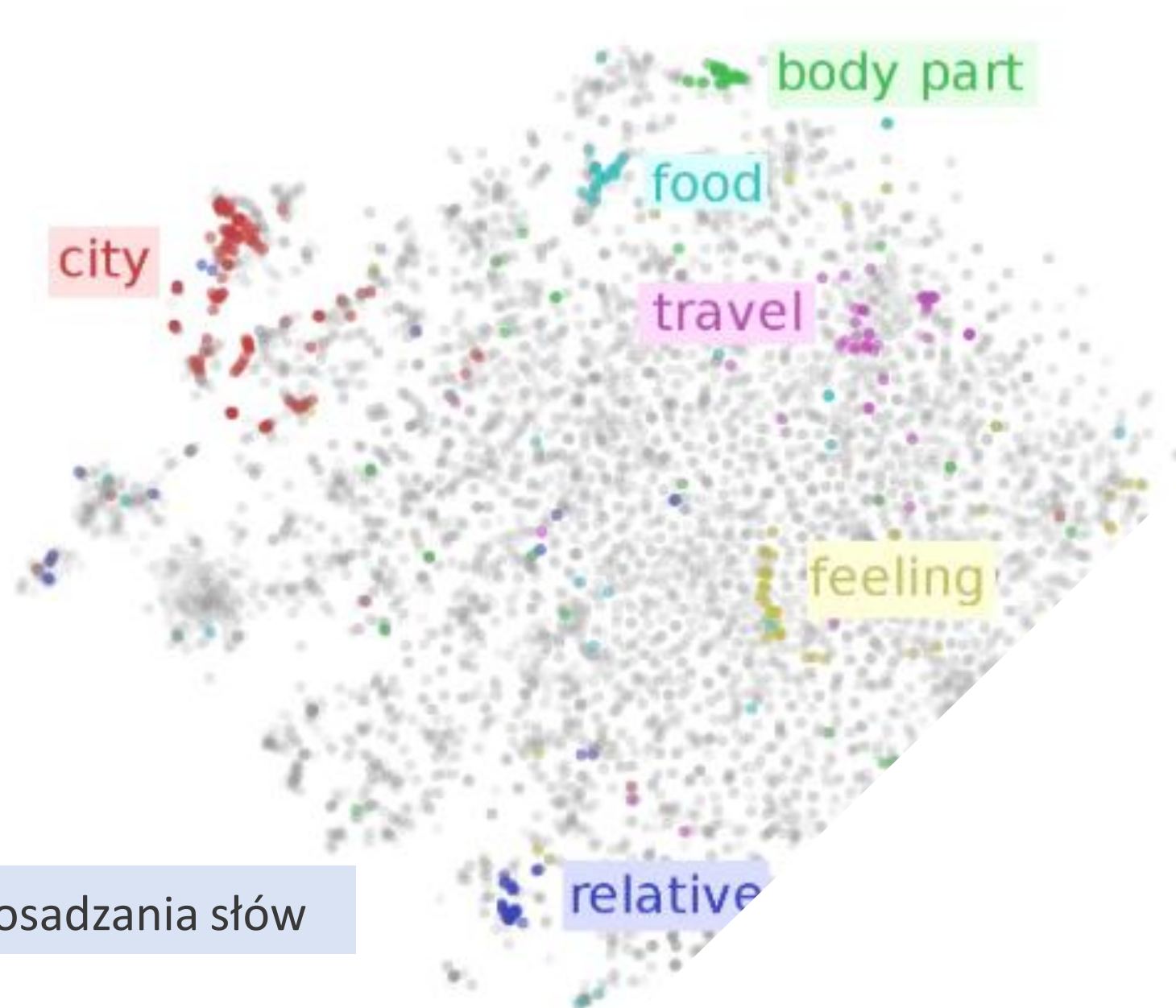
Dane przygotowane jako tokeny (pnie, chunks).

Do tokenów stosujemy osadzanie słów (word embedding)- tokeny reprezentowane są jako wielowymiarowe wektory.



Osadzanie słów to język AI. Każde słowo jest reprezentowane jako wielowymiarowy wektor, który zawiera jego znaczenie semantyczne w oparciu o kontekst w danych uczących. Te wektory pozwalają AI zrozumieć relacje i podobieństwa między słowami, zwiększając zrozumienie i wydajność modelu.

Wizualizacje osadzania słów (PCA, tSNE)



Algorytmy Word2vec i GloVe do osadzania słów

LLM-jak działają?



Dane

GPT-3, jeden z największych LLM-ów jest trenowany na 570 GB danych tekstowych. Mniejsze LLM-y mogą potrzebować mniej – może 10-20 GB lub nawet 1 GB gigabajtów – ale to i tak dużo. Tekst czyścimy i dzielimy na pnie- tokeny



Model

Np. GPT
GPT3 ma
175miliarów
parametrów

Cel: dla danego ciągu tokenów
przewidywanie następnego tokenu

- Model bazowy przewiduje następny token.
- Nie jest w stanie odpowiedzieć na pytania, itp.

Przewidywanie następnego tokenu- next token prediction

Surowy tekst

Gra WOT jest wspaniała

Tokenizacja i osadzenie

Gra

WOT

jest

Duży model językowy LLM

Next token prediction

Gra

WOT

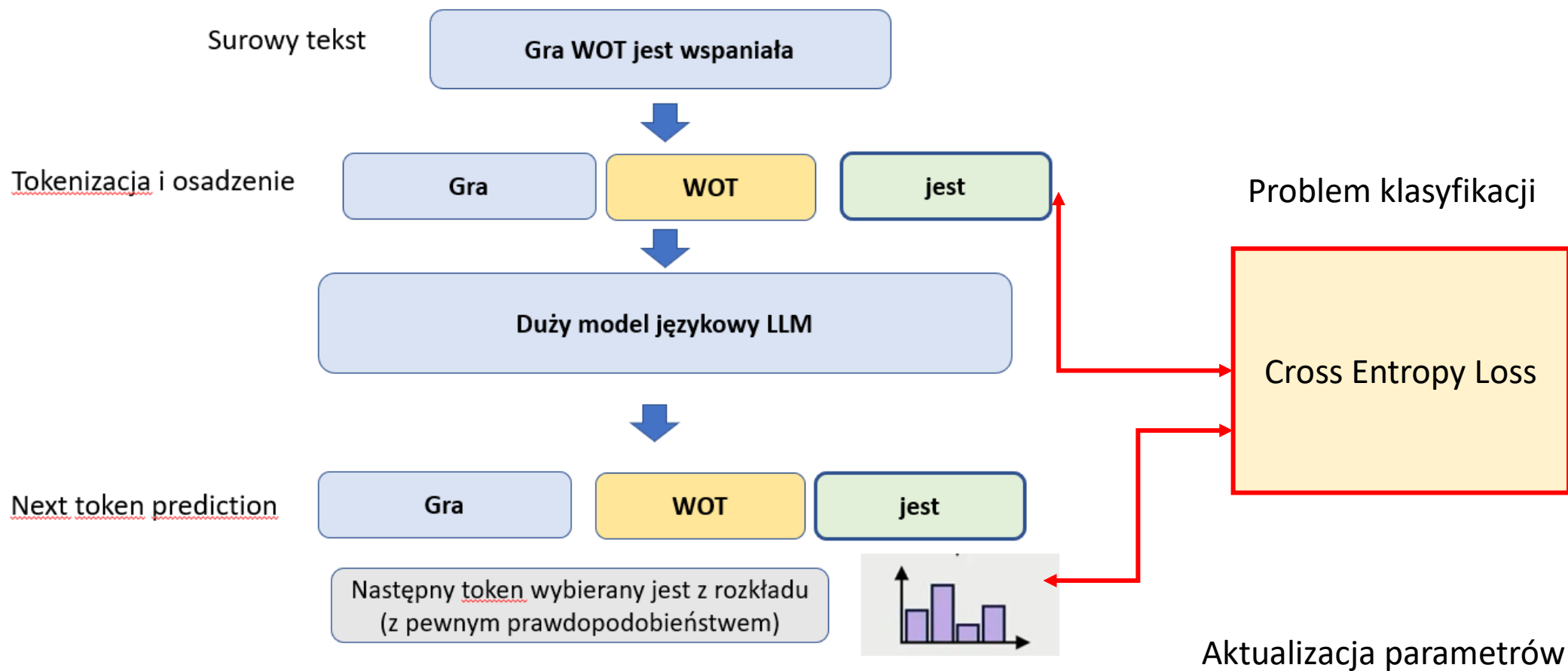
jest

wspaniała

Następny token wybierany jest z rozkładu
(z pewnym prawdopodobieństwem)



Przewidywanie następnego tokenu- next token prediction



Self-supervised learning (SSL) –uczenie z samonadzorem

Self-supervised learning w LLM polega na uczeniu modeli języka poprzez przewidywanie następnego tokenu w tekście. Ten sam tekst jest inputem i outputem. Model uczy się bez etykiet, tworząc je z samych danych wejściowych.



Polecam

MIT 6.S191 (Liquid AI): Large Language Models

https://www.youtube.com/watch?v=_HfdncCbMOE&t=487s

Wykład pojawił się 21 kwietnia 2025 r. Nie ma jeszcze z 2026 roku.

Wykładowca: **Maxime Labonne** (Liquid AI)

<https://github.com/mlabonne/llm-course>

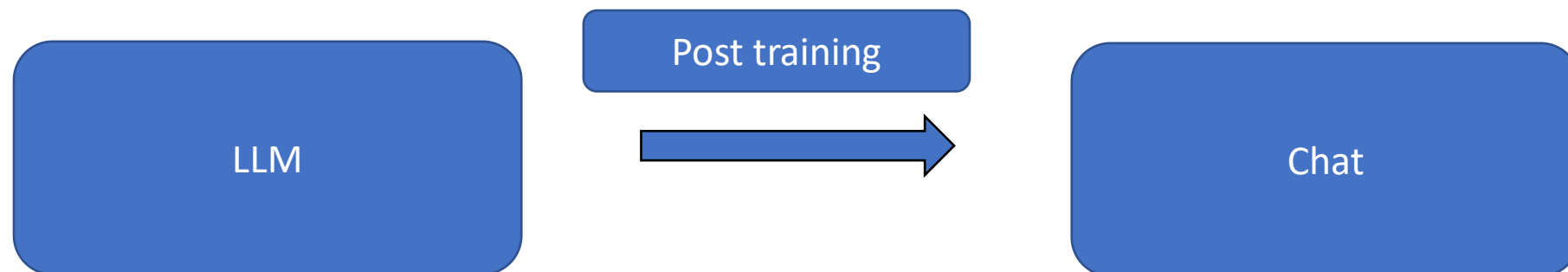


- 00:01 - Introduction to LLM Post-Training
- 01:47 - Preference Alignment and Data Set Considerations
- 03:28 - When and Why to Fine-Tune: Use Cases and Data Quality
- 10:28 - Data Complexity, Formats, and Generation Pipelines
- 15:47 - Preference Data Generation and Chat Templates
- 19:11 - Fine-Tuning Libraries and Techniques
- 24:30 - Training Parameters, Experiment Monitoring, and Model Merging
- 33:42 - Model Merging Case Study and Evaluation Challenges
- 41:10 - Evaluation Methods and Future Trends: Test-Time Compute Scaling

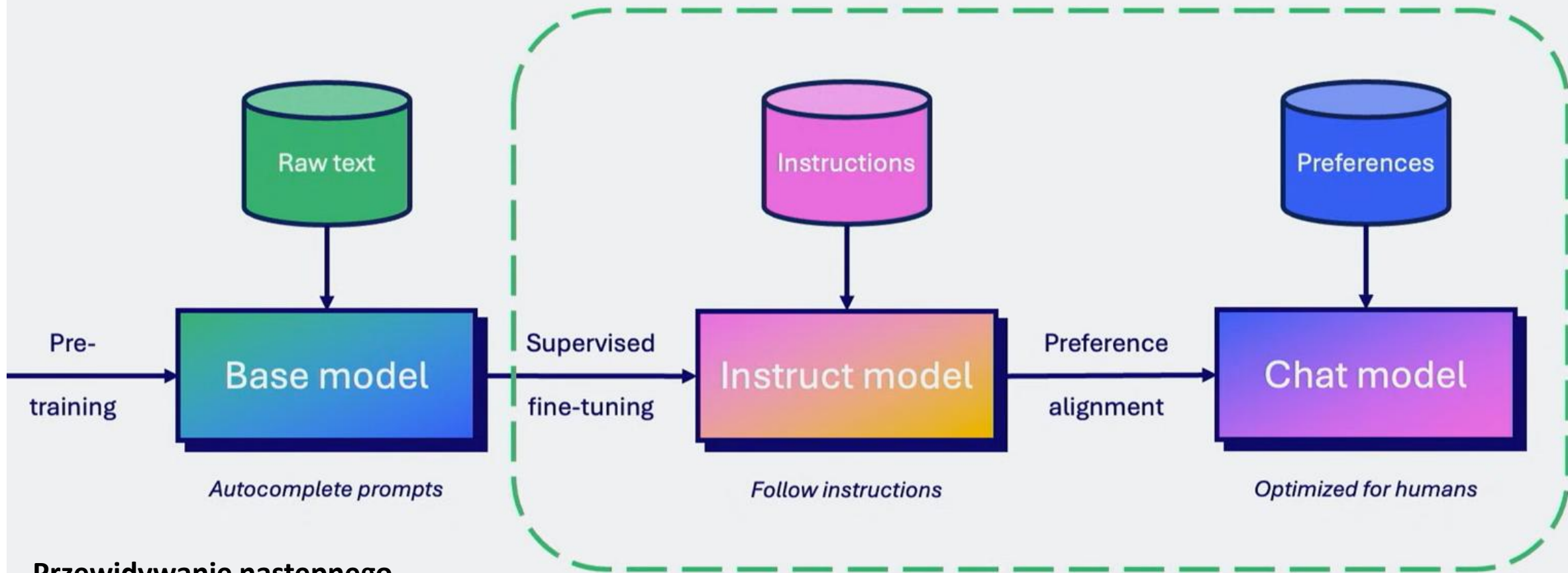
Post training

Post-training to zbiór technik, które przekształcają model bazowy w użyteczną aplikację:

- uczą model **zachowania**
- dopasowują go do **intencji użytkownika**
- poprawiają **bezpieczeństwo, spójność i użyteczność**



Post-Training



Przewidywanie następnego tokenu- next token prediction

Uczymy model interpretowania instrukcji i odpowiadania na pytania

Uczymy model prawidłowych odpowiedzi

Główne techniki post-trainingu

- **Supervised Fine-Tuning (SFT)** –model uczy się na parach pytanie-idealna odpowiedź stylu odpowiedzi, klarowności i struktury.
- **RLHF – Reinforcement Learning from Human Feedback** -model generuje odpowiedzi, które oceniają ludzie-model z nagrodą
- **Safety tuning / alignment tuning** –model uczy się odmawiać niebezpiecznych poleceń, unikać halucynacji, reagować neutralnie na kontrowersyjne tematy
- **Instruction tuning** –model uczy się interpretować instrukcje i reagować na polecenia

ChatGPT (<https://openai.com/blog/chatgpt/>)

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain reinforcement learning to a 6 year old.

A labeler demonstrates the desired output behavior.

We give treats and punishments to teach...

This data is used to fine-tune GPT-3.5 with supervised learning.

SFT

Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

Explain reinforcement learning to a 6 year old.

A In reinforcement learning the agent is...
B Explain rewards...
C In machine learning...
D We give treats and punishments to teach...

A labeler ranks the outputs from best to worst.

D > C > A > B

This data is used to train our reward model.

RM
D > C > A > B

Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.

Write a story about otters.

The PPO model is initialized from the supervised policy.

PPO

The policy generates an output.

Once upon a time...

The reward model calculates a reward for the output.

RM

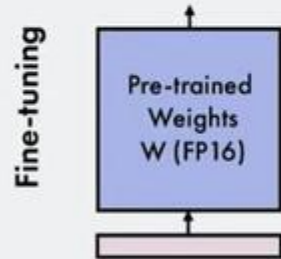
The reward is used to update the policy using PPO.

r_k

Supervised Fine-Tuning (SFT)

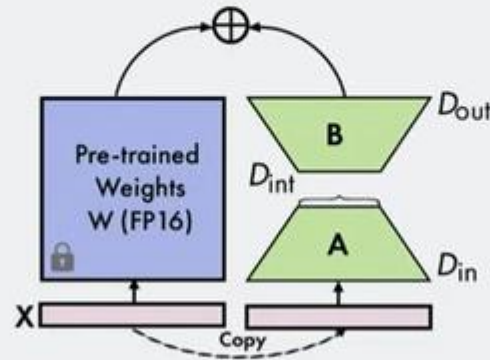
SFT techniques

Full Fine-Tuning 16-bit precision



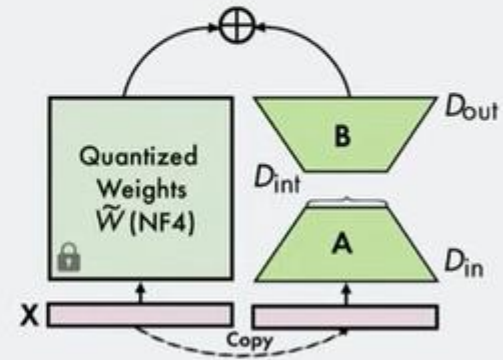
- ✓ Maximizes quality
- ✗ Very high VRAM usage

LoRA 16-bit precision



- ✓ Fastest training
- ✗ High VRAM usage

QLoRA 4-bit precision

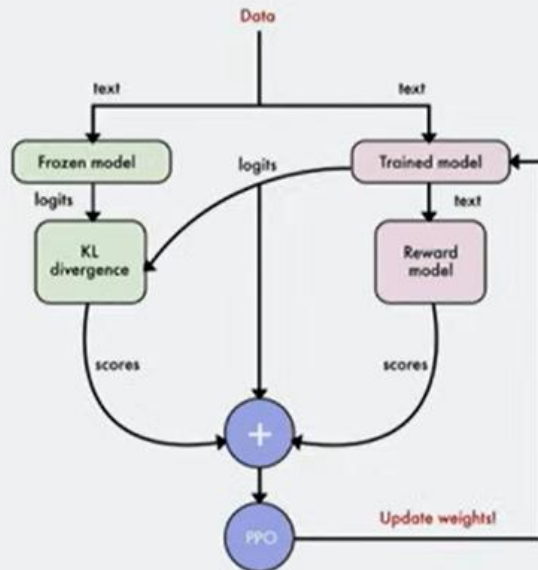


- ✓ Low VRAM usage
- ✗ Degrades performance

Metody uczenia ze wzmacnianiem

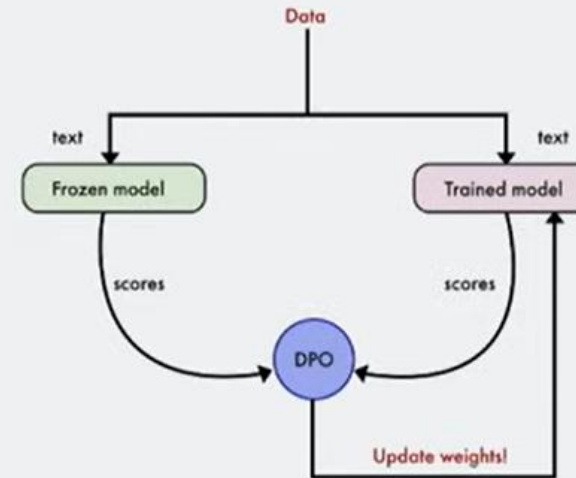
Preference alignment techniques

Proximal Policy Optimization



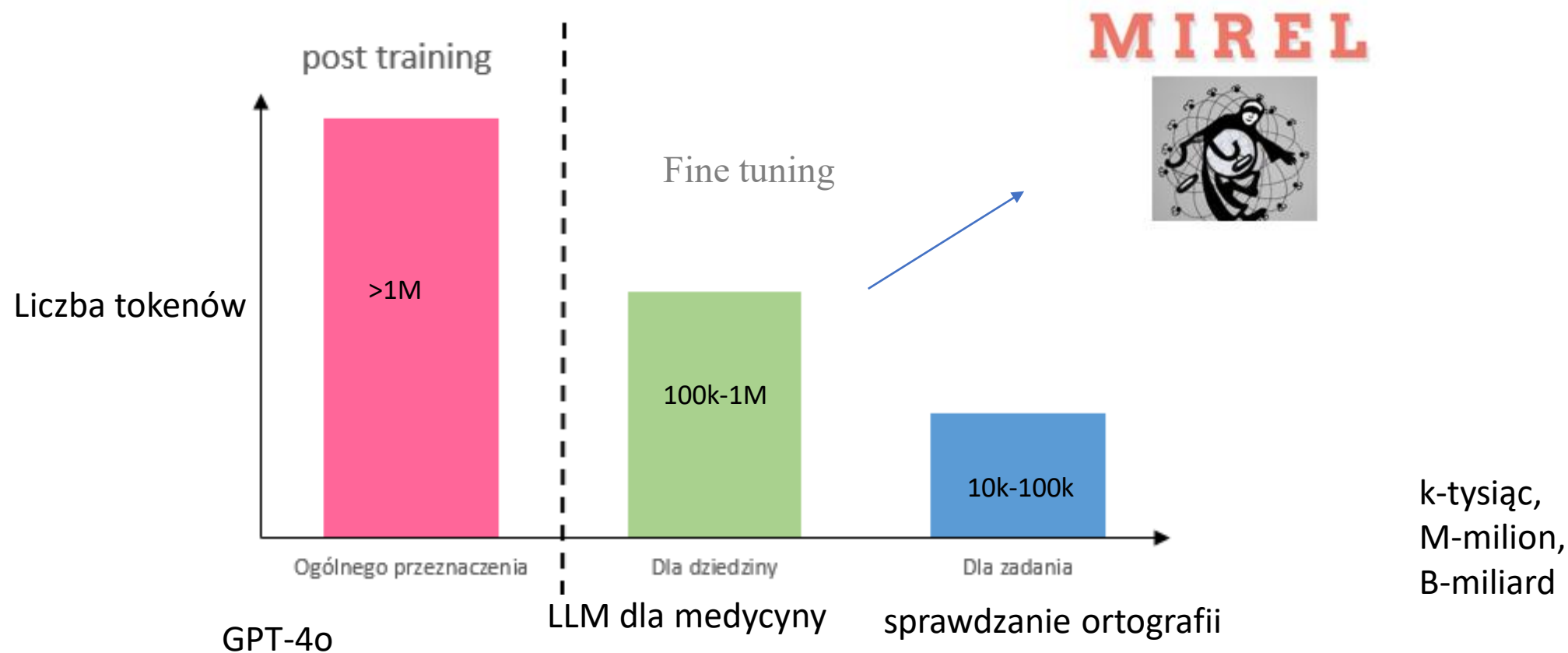
- ✓ Maximizes quality
- ✗ Very expensive & complex

Direct Preference Optimization



- ✓ Fast and cheap to use
- ✗ Lower quality

Post training-przekształcenie modelu bazowego w użyteczną aplikację



**Generative Pre-trained Transformer
4 Omni (GPT-4o)**

Przykład

MIREL

Kontekst

Tworzenie ustaw, orzeczeń sądów w sprawach lub ogólnie tekstów prawnych jest jedną z kluczowych prerogatyw sektora publicznego. Umiejętność rozumowania na podstawie danych prawnych była od kilku lat głównym obszarem badań.

Praktycznym przykładem projektu, który wdrożył NLP na publicznych danych prawnych, jest MIREL (MINing and REasoning with Legal texts). Celem tej finansowanej przez UE inicjatywy było przetłumaczenie tekstów prawnych na formalne reprezentacje, które można wykorzystać do sprawdzania norm, sprawdzania zgodności i wspomagania decyzji.



Querying norms



Compliance checking



Decision support

Figure: Use cases in MIREL

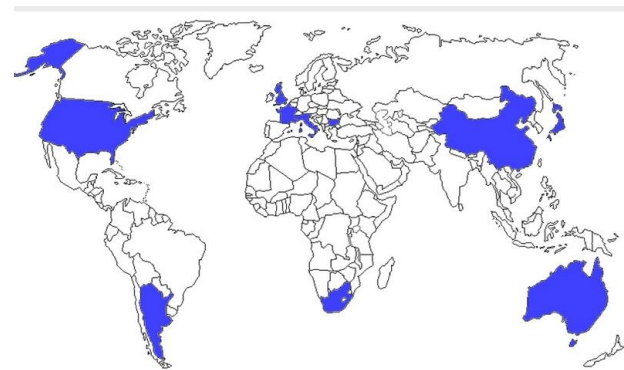
MIREL



D REASONING WITH LEGAL TEXTS.

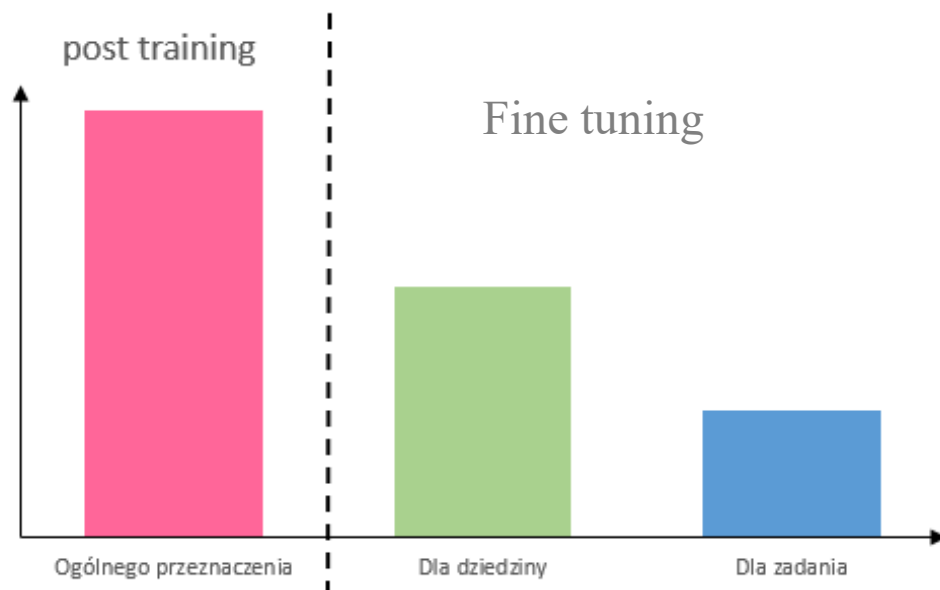
2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No L Consortium includes 16 members from 11 countries, at least one for each continent. 8 beneficiaries (within Europe) and 8 partners (outside Europe).

IIREL beneficiaries are key players in the communities of Deontic, AI & Law, and the Semantic Web, communities that have traditionally been strong in Europe. The . consortium brings these scientists together with researchers with expertise traditionally lacking in Europe, such as norm and argument mining (Argentina, Japan and), description logic for reasoning about legal ontologies (South Africa), natural language semantics of deontic modals (USA), and the complexity analysis of regulatory liance (Australia).



Fine tuning-dokładne dostrajanie

- Grammarly
- np. budowa chatbota
- można zbudować model dla konkretnej dziedziny, medycyny, prawa, finansów.



OpenAI Developer Community

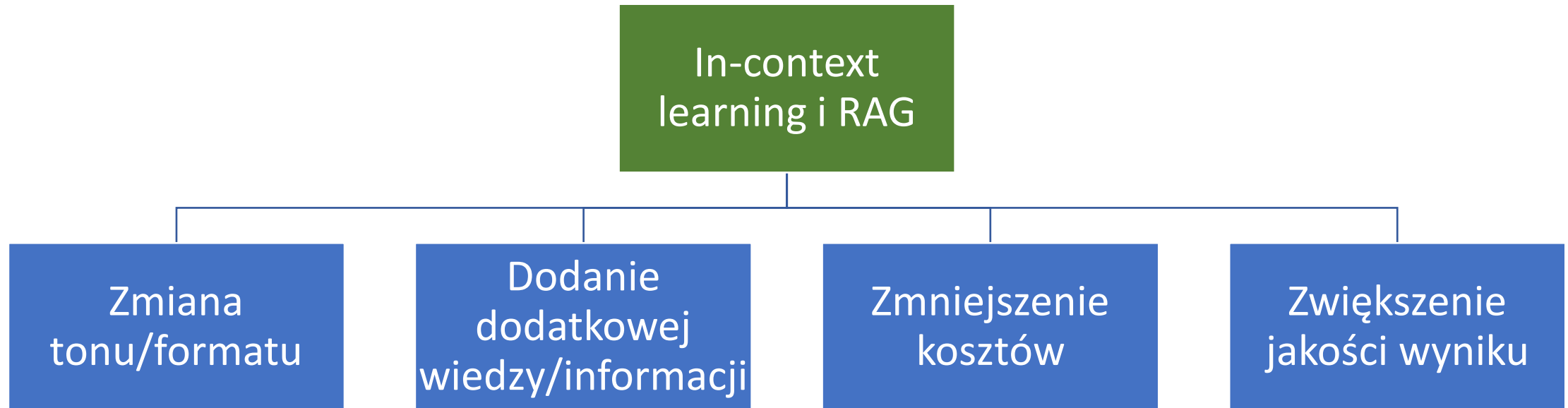


dsco

I'm currently exploring fine-tuning with **GPT-4-Real-Time**.

My goals are to adjust not just the content, but also the **style, tone**, and even **vocalization features** such as **pauses** and **word emphasis** during responses.

Kiedy stosujemy dostrajanie?



Incontext learning i RAG

Uczenie się w kontekście (ICL) to technika, w której demonstracje zadań są zintegrowane z odpowiedzią w formacie języka naturalnego.

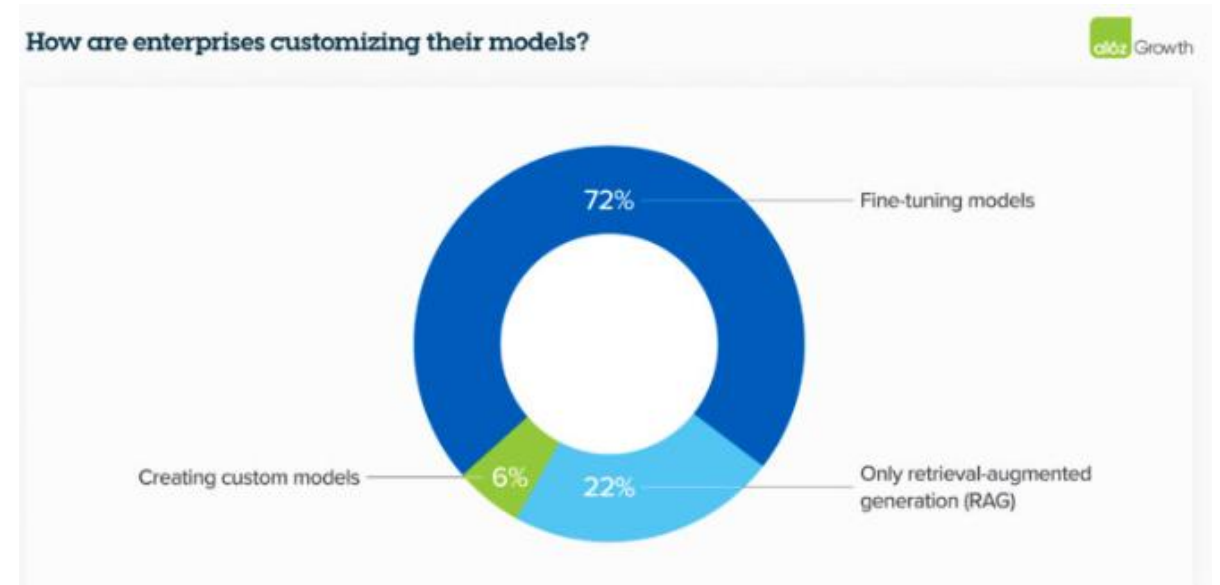
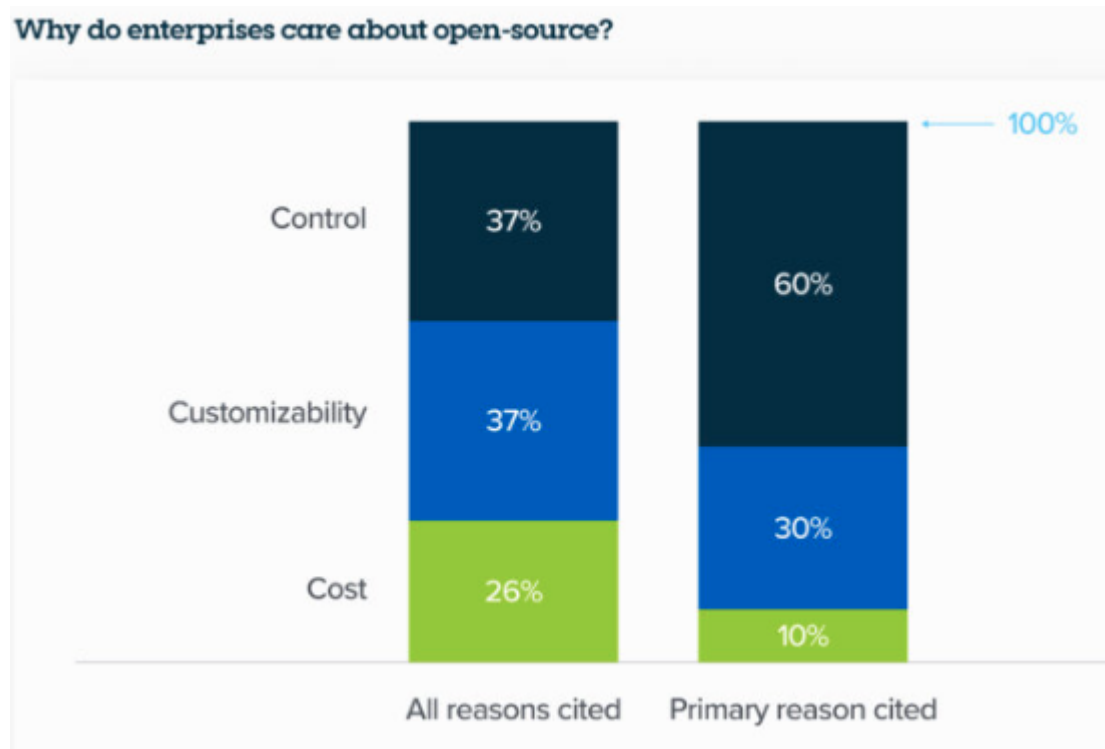
Podejście to pozwala wstępnie wytrenowanym LLM na rozwiązywanie nowych zadań bez konieczności dostrajania modelu.

Retrieval augmented generation (RAG) to architektura, która poprawia wydajność aplikacji sztucznej inteligencji (AI) poprzez wykorzystanie zewnętrznych baz wiedzy.

RAG wykorzystuje techniki pobierania i wstępnego przetwarzania, aby wykorzystać dane ze źródeł zewnętrznych, takich jak bazy wiedzy, bazy danych i strony internetowe. Po znalezieniu przez model danych w celu wygenerowania najlepszego wyniku, poddaje on informacje wstępnemu przetwarzaniu, które obejmuje usuwanie słów stop, tokenizację i stemming.

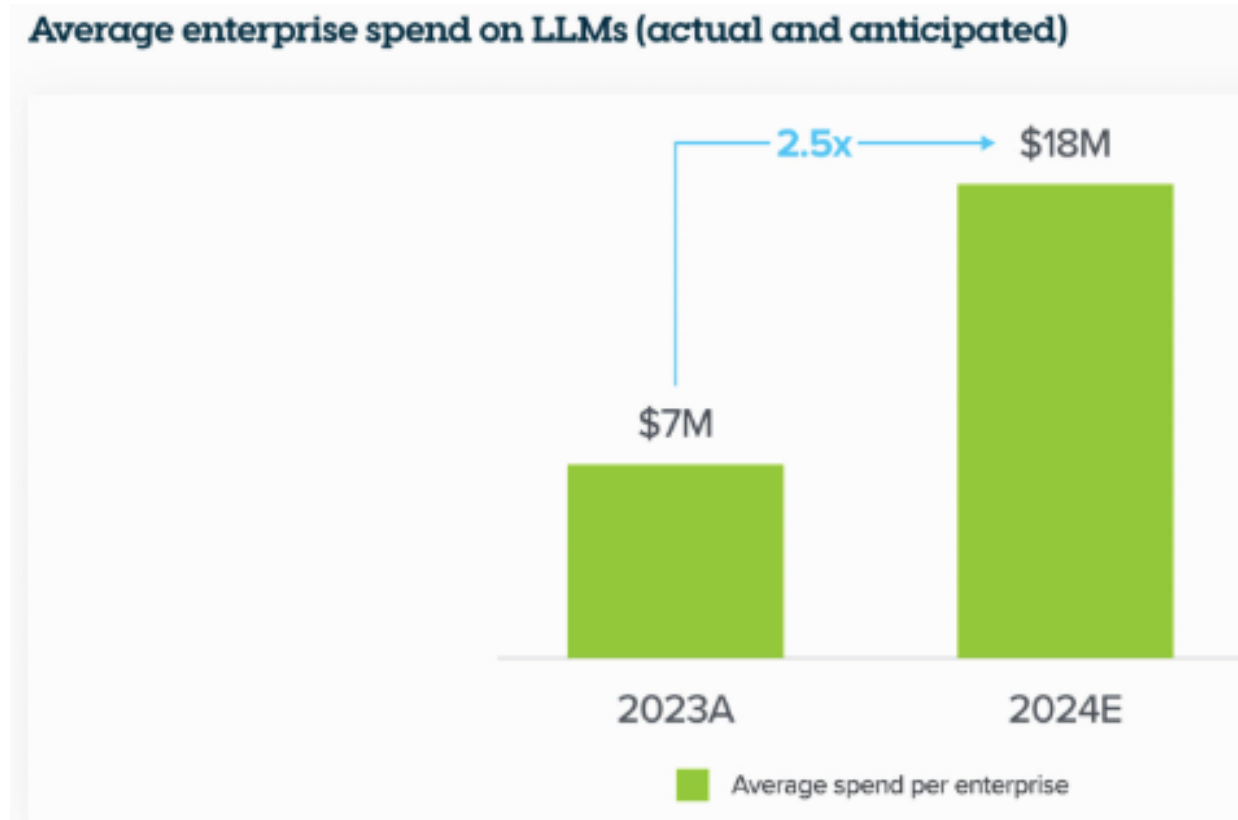
- <https://www.lakera.ai/blog/what-is-in-context-learning>

Po co Fine-Tuning (dokładne dostrajanie)



<https://a16z.com/generative-ai-enterprise-2024/>

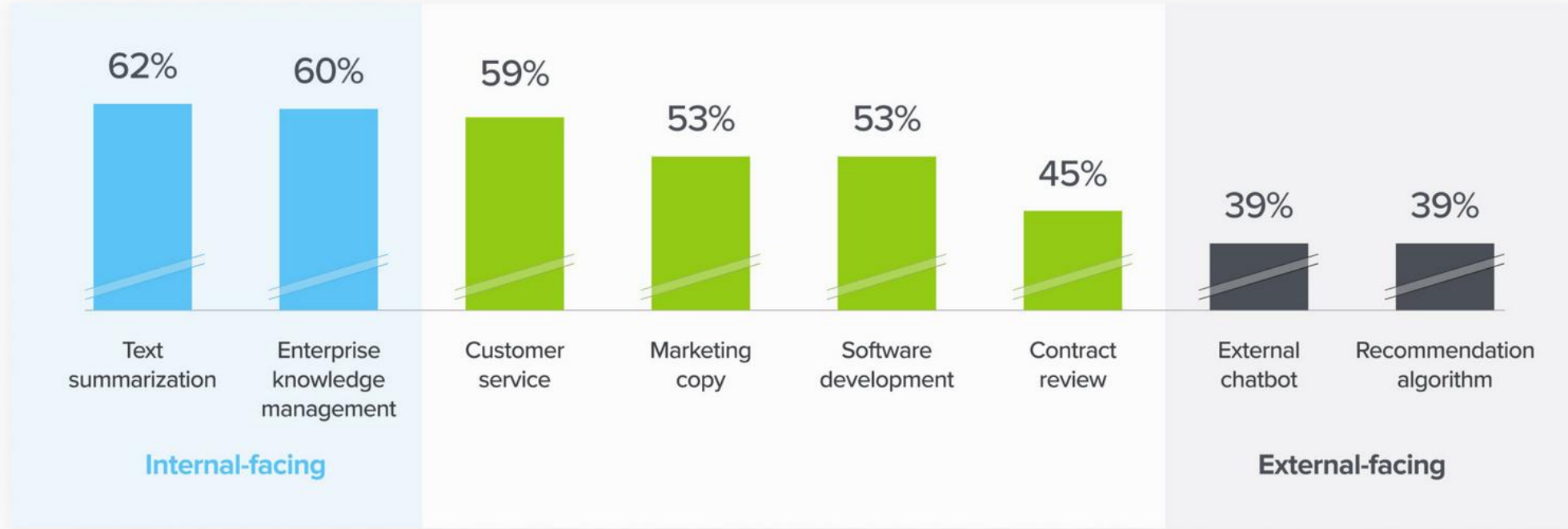
<https://a16z.com/generative-ai-enterprise-2024/>



How willing are enterprises to use LLMs for different use cases?



(% of enterprises experimenting with given use case who have deployed to production)



Source: a16z survey of 70 enterprise AI decision makers

Dostrajanie



😊 Dataset: [huggingface.co/datasets/mlabo...](https://huggingface.co/datasets/mlabonne/open-perfectblend)
📄 Paper: arxiv.org/abs/2409.20370
Przetłumacz wpis

Open-PerfectBlend

5.1 Supervised Fine-Tuning

The foundational model we have chosen is the LLaMA-3.0-70B pre-trained checkpoint. We independently perform SFT using an open-source dataset to establish the initial policy, denoted as π_0 . For all preference pair datasets listed below we only use positive samples in SFT. We utilize the following datasets for the tasks under consideration:

- **General chat:** LMSys-55k (Chiang et al., 2024), UltraChat (Ding et al., 2023)
- **Instruction following:** Llama 3.0 70B instruct model synthetic instruction following dataset
- **Math/Code Reasoning:** Orca-Math Mitra et al. (2024), MetaMath (Yu et al., 2023), Evol-CodeAlpaca (Luo et al., 2023), UltraFeedback (Cui et al., 2023), UltraInteract (Yuan et al., 2024a)
- **Harmful Intent:** Human annotated safety dataset

The training is carry out for 2 epoches with a learning rate of 10^{-5} . A cosine schedule is employed, the global batchsize is set to 128 with minimum rate 0.1 and warm-up steps 200. The detail of how we obtain synthetic instruction following dataset and safety dataset SFT can be found in Appendix A.

Dataset	# Samples
meta-math/MetaMathQA	395,000
openbmb/UltraInteract_sft	288,579
HuggingFaceH4/ultrachat_200k	207,865
microsoft/orca-math-word-problems-200k	200,035
HuggingFaceH4/ultrafeedback_binarized	187,405
theblackcat102/evol-codealpaca-v1	111,272
Post-training-Data-Flywheel/AutoF-instruct-61k	61,492
mlabonne/lmsys-arena-human-preference-55k-sharegpt	57,362

1.42M samples, Apache 2.0

Open-PerfectBlend is an open-source reproduction of the instruction dataset introduced in the paper ["The Perfect Blend: Redefining RLHF with Mixture of Judges"](https://arxiv.org/abs/2409.20370). It's a solid general-purpose instruction dataset with chat, math, code, and instruction-following data.

<https://huggingface.co/datasets/mlabonne/open-perfectblend>

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao^{*,1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

¹Department of Computer Science, Princeton University

²Google Research, Brain team

¹{shunyuy, karthikn}@princeton.edu

²{jeffreyzhao, dianyu, dunan, izhak, yuancao}@google.com

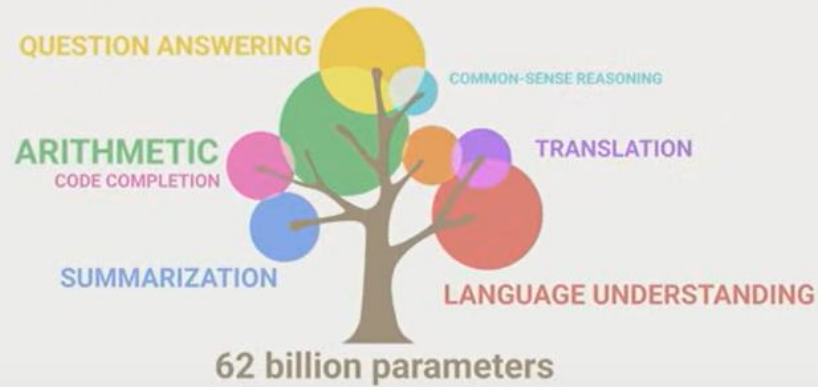
Toolformer: Language Models Can Teach Themselves to Use Tools

Timo Schick Jane Dwivedi-Yu Roberto Dessì[†] Roberta Raileanu
Maria Lomeli Luke Zettlemoyer Nicola Cancedda Thomas Scialom

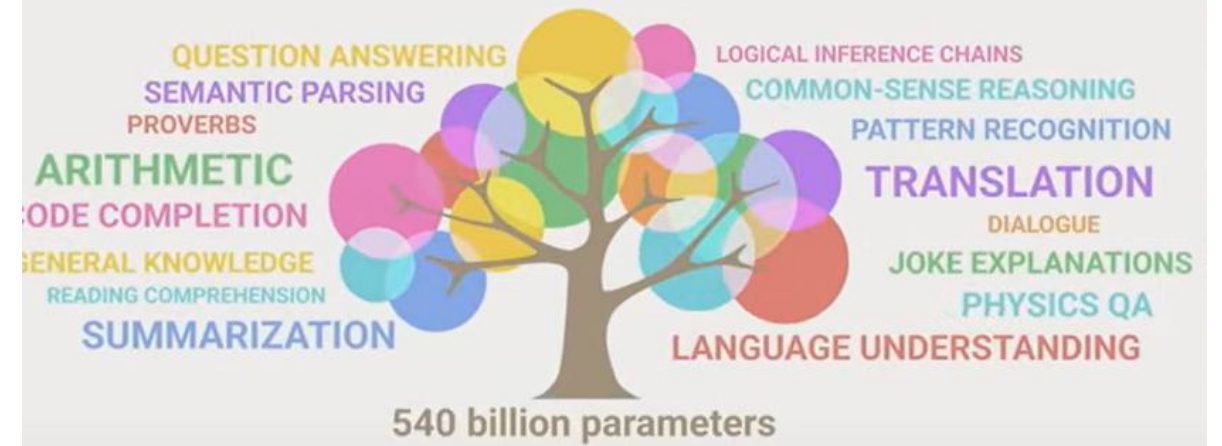
Meta AI Research [†]Universitat Pompeu Fabra

- MIT Deep learning 6.S191

Emergent Abilities: Towards Intelligence



Emergent Abilities: Towards Intelligence



Z wykładu MIT

Recommended fine-tuning libraries



TRL

HF's library, most up-to-date in terms of algorithms



Axolotl

Additional features and reusable YAML configurations



Unsloth

Efficient single GPU fine-tuning with useful utilities

<https://huggingface.co/docs/trl/index>



TRL - Transformer Reinforcement Learning

TRL is a full stack library where we provide a set of tools to train transformer language models with methods like Supervised Fine-Tuning (SFT), Group Relative Policy Optimization (GRPO), Direct Preference Optimization (DPO), Reward Modeling, and more. The library is integrated with 🧠 [transformers](#).



Hugging Face

Search

Transformers

Search documentation Ctrl+K

V4.51.3

EN



143,938

```
from transformers import AutoTokenizer, AutoModel
import torch
import torch.nn.functional as F
```



The AI community building the future.

The platform where the machine learning community
collaborates on models, datasets, and applications.

Explore AI Apps

or [Browse 1M+ models](#)



Datasets



huggingface.co/docs/hub/repositories

Repositories - Introduction - Getting Started - Repository Settings - Pull requests and Discussions - Notifications - Collections - Webhooks - Storage Backends - Next Steps - Licenses	Models - Introduction - The Model Hub - Model Cards - Gated Models - Uploading Models - Downloading Models - Libraries - Tasks - Widgets - Inference API - Download Stats	Datasets - Introduction - Datasets Overview - Dataset Cards - Gated Datasets - Uploading Datasets - Downloading Datasets - Libraries - Dataset Viewer - Download Stats - Data files Configuration
Spaces - Introduction - Spaces Overview - Gradio Spaces - Streamlit Spaces - Static HTML Spaces - Docker Spaces - Embed your Space - Run with Docker - Reference - Changelog - Advanced Topics - Sign in with HF	Other - Organizations - Enterprise Hub - Billing - Security - Moderation - Paper Pages - Search - Digital Object Identifier (DOI) - Hub API Endpoints - Sign in with HF - Contributor Code of Conduct - Content Guidelines	

 **Hugging Face**

Hub Ctrl+K

EN 387

 **Hugging Face Hub**

Repositories

- Getting Started with Repositories
- Repository Settings
- Pull Requests & Discussions
- Notifications
- Collections
- Webhooks
- Notebooks
- Storage Backends
- Storage Limits
- Next Steps
- Licenses

O'REILLY®

Natural Language Processing with Transformers

Building Language Applications
with Hugging Face



Lewis Tunstall,
Leandro von Werra
& Thomas Wolf

Revised
Edition

Using 🤗 transformers at Hugging Face

🤗 transformers is a library maintained by Hugging Face and the community, for state-of-the-art Machine Learning for Pytorch, TensorFlow and JAX. It provides thousands of pretrained models to perform tasks on different modalities such as text, vision, and audio. We are a bit biased, but we really like 🤗 transformers!

Exploring 🤗 transformers in the Hub

There are over 25,000 transformers models in the Hub which you can find by filtering at the left of [the models page](#).

You can find models for many different tasks:

- Extracting the answer from a context ([question-answering](#)).
- Creating summaries from a large text ([summarization](#)).
- Classify text (e.g. as spam or not spam, [text-classification](#)).
- Generate a new text with models such as GPT ([text-generation](#)).
- Identify parts of speech (verb, subject, etc.) or entities (country, organization, etc.) in a sentence ([token-classification](#)).
- Transcribe audio files to text ([automatic-speech-recognition](#)).
- Classify the speaker or language in an audio file ([audio-classification](#)).
- Detect objects in an image ([object-detection](#)).
- Segment an image ([image-segmentation](#)).
- Do Reinforcement Learning ([reinforcement-learning](#))!

You can try out the models directly in the browser if you want to test them out without downloading them thanks to the in-browser widgets!

Developing LLM Powered Applications: Llama 2 on Azure (5/n)

[Madhusudhan Konda](#)



<https://mkonda007.medium.com/developing-llm-powered-applications-llama-2-on-azure-5-n-1bd71672fe4c>

Deploying LLM model in Azure AI Studio in 5 minutes



Kosta Malsev · [Follow](#)

3 min read · Aug 30, 2024

- <https://kostya-malsev.medium.com/deploying-llm-model-in-azure-ai-studio-in-5-minutes-cfe0c5a4c838>

Małe modele językowe SLM

Małe modele językowe są kompaktowymi, wysoce wydajnymi wersjami masywnych dużych modeli językowych, o których tak wiele słyszeliśmy. Modele LLM, takie jak GPT-4o, mają setki miliardów parametrów, ale SML używają znacznie mniej - zazwyczaj od milionów do kilku miliardów.

SMALL LANGUAGE MODELS:
SURVEY, MEASUREMENTS, AND INSIGHTS

Zhenyan Lu^{★◇†}, Xiang Li^{★†}, Dongqi Cai[★], Rongjie Yi[★], Fangming Liu[◇], Xiwen Zhang[♥],
Nicholas D. Lane[♡], Mengwei Xu[★]

[★]Beijing University of Posts and Telecommunications (BUPT)
[◇]Peng Cheng Laboratory
[♥]Helixon Research
[♡]University of Cambridge

Cechy SLM

Wydajność: SLM nie potrzebują ogromnej mocy obliczeniowej, której wymagają LLM. Dzięki temu świetnie nadają się do użytku na urządzeniach o ograniczonych zasobach, takich jak smartfony, tablety lub urządzenia IoT.

Dostępność: Osoby z ograniczonym budżetem mogą wdrożyć SLM bez konieczności posiadania zaawansowanej infrastruktury. Są one również dobrze dostosowane do wdrożeń lokalnych, w których prywatność i bezpieczeństwo danych są bardzo ważne, ponieważ nie zawsze polegają na infrastrukturze opartej na chmurze.

Cechy SLM

Personalizacja: SLM są łatwe do dostrojenia. Ich mniejszy rozmiar oznacza, że mogą szybko dostosować się do niszowych zadań i wyspecjalizowanych domen. Dzięki temu dobrze nadają się do konkretnych zastosowań, takich jak obsługa klienta, opieka zdrowotna lub edukacja.

Szybsze wnioskowanie: SLM mają szybszy czas reakcji, ponieważ mają mniej parametrów do przetworzenia. Sprawia to, że są one idealne do zastosowań w czasie rzeczywistym, takich jak chatboty, wirtualni asystenci lub dowolny system, w którym szybkie decyzje są niezbędne. Nie trzeba czekać na odpowiedzi, co jest świetnym rozwiązaniem w środowiskach, w których małe opóźnienia są koniecznością.

2.1 Overview of SLMs

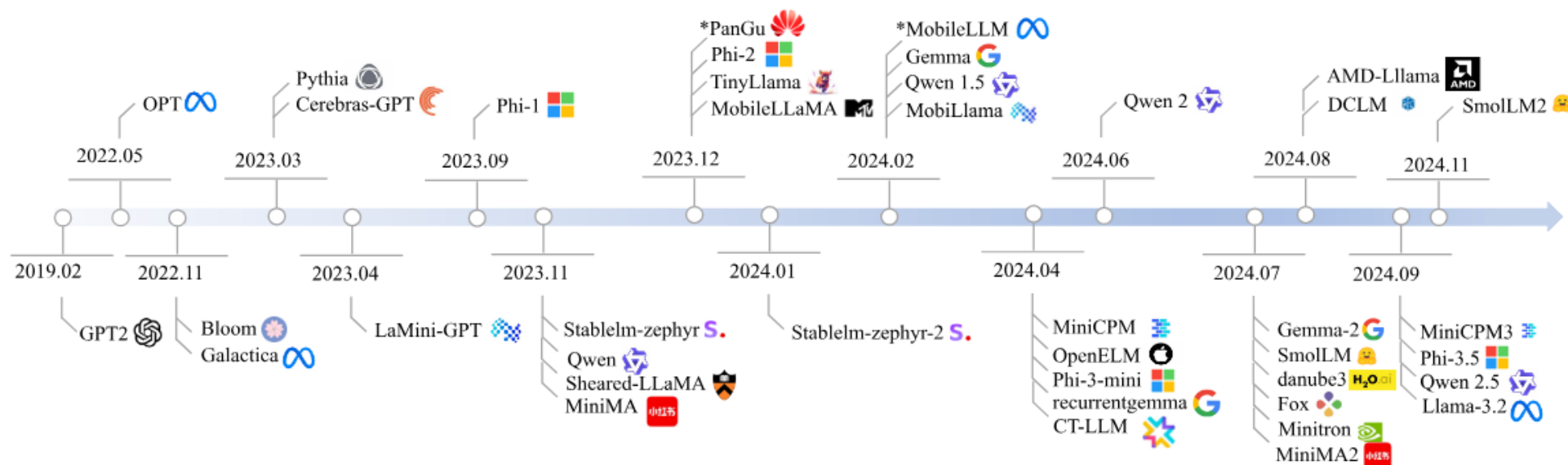


Figure 1: An overview of SLMs. * indicates the models are not open-sourced so will not be benchmarked.

SMALL LANGUAGE MODELS: SURVEY, MEASUREMENTS, AND INSIGHTS

Ćwiczenia 10-Dostrajanie SLM

1. **Prywatność** – Dostrajanie własnego modelu oznacza, że możesz go uruchamiać na swoim sprzęcie i nie musisz wysyłać żadnych danych do obcej firmy.
2. **Wysoka wydajność przy mniejszym obciążeniu** –wiele można osiągnąć dzięki dostosowanemu, mniejszemu modelowi przeznaczonemu do konkretnego zadania.
3. **Możliwość pracy w trybie offline** – Jeśli masz mniejszy model niestandardowy, możesz wdrożyć go na sprzęcie brzegowym, takim jak urządzenia mobilne, i uruchamiać go nawet bez połączenia z internetem
4. **Uruchamianie w trybie wsadowym**– Mniejszy model często może działać z wieloma próbkami jednocześnie (tryb wsadowy), a dzięki silnikom wnioskowania (inference engines), takim jak vLLM, często można go używać na setkach lub tysiącach próbek na sekundę.

Pozwala to na realizację zadań wnioskowania na dużą skalę.

5. **Oszczędność kosztów**— Po dostrojeniu małego modelu do wykonywania określonego zadania można go uruchamiać wielokrotnie bez ponoszenia kosztów API.

Pamiętaj: modele LLM działają na zasadzie „token in, token out”. Jeśli dysponujesz odpowiednim zestawem tokenów, możesz stworzyć własny, wyspecjalizowany model.

